Truncate or Not? A Scaling Study of Looped Transformers with Truncated Backpropagation

Xuyuan Xiong[1,*], **Siyu Chen**[2,*], **Zihan Shao**[2], **Yue Su**[1], **Weiyang Jin**[1], **Ping Luo**[1], **Zhuoran Yang**[2,†], **Xihui Liu**[1,†]

[1]MMLab, The University of Hong Kong [2]Yale University

[*]Equal contribution [†]Corresponding authors

Looped transformers reuse a shared block of layers to increase computational depth without increasing the number of trainable parameters. Training uses backpropagation through time (BPTT), either through all loops or through only the last part of the recurrence. Truncating backpropagation saves training compute and activation memory, but can reduce the gains from looping. Across nearly 600 training runs spanning over three orders of magnitude in training tokens per parameter, we study how model size, training tokens, and forward and backward loop counts jointly determine loss, and fit a unified scaling law that captures the effects of looping and truncation through *effective parameters*. Despite the training-compute savings of truncation, our fitted law favors **full BPTT** when forward and backward loop counts are jointly optimized. In matched-compute comparisons spanning 58M–919M non-embedding parameters and up to 300 training tokens per parameter, full BPTT through fewer loops consistently outperforms truncated settings of the same compute. Using this law, we further show that at fixed model size, compute-optimal training increases both the loop count and the training-token budget as compute grows. When model size is also free, the law identifies two fully trained loops as the best looped setting. Experimentally, this setting achieves lower loss than the compute-optimal non-looped model with about **27% fewer non-embedding parameters** and only 8% more training compute.

Date: October 16, 2026

Code: https://github.com/xjxyys/Truncate-or-Not

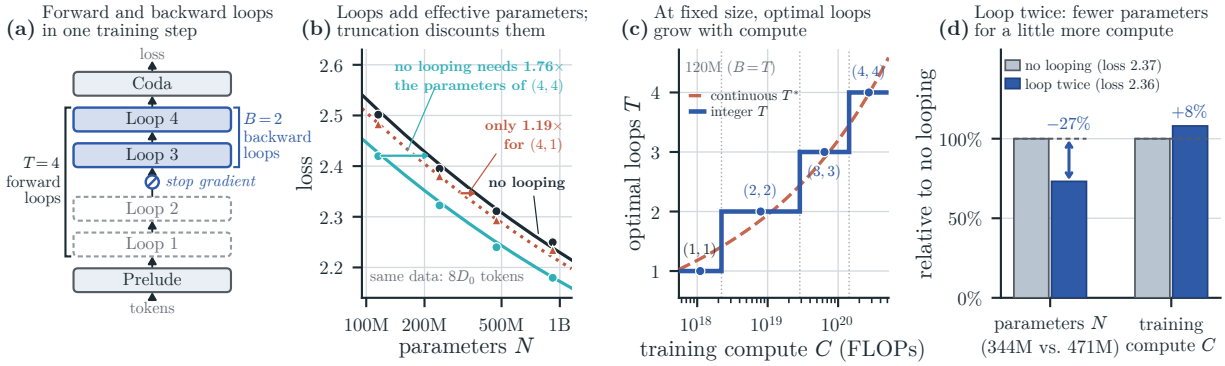


Figure 1 | **Overview of looped transformer scaling with truncated BPTT. (a)** Step for $(T, B) = (4, 2)$ (T forward loops, last B trained). **(b)** At $8D_0$ tokens, loops add effective parameters while truncation discounts them (§4; lines: law, markers: runs). **(c)** At fixed size (120M), optimal T grows with compute under full BPTT (§6.1). **(d)** $(2, 2)$ (344M) beats $(1, 1)$ (471M, 2.1×10^{19} FLOPs) in loss (2.36 vs. 2.37 nats) with 27% fewer parameters and 8% more compute (§6.3).

1 Introduction

Looped transformers have attracted attention for combining strong performance with efficient use of parameters and training memory. By repeatedly applying a shared block of layers, they increase computational depth without increasing the number of trainable parameters (Dehghani et al., 2019; Geiping et al., 2025). To reduce the cost of training these repeated blocks, several looped language models use truncated backpropagation through time (BPTT), propagating gradients only through the final loops while executing the earlier loops without tracking gradients (Geiping et al., 2025; Huang et al., 2026; McLeish et al., 2025; Prairie et al., 2026; Wang et al., 2026a). Truncation introduces a new tradeoff: it saves backward computation and activation memory during training but can weaken the gains from looping (Schwethelm et al., 2026). To achieve the best performance under a fixed training compute budget, we must therefore decide how to divide compute among forward loops, backward loops, model size, and training tokens.

Existing scaling studies examine prescribed backpropagation recipes rather than jointly modeling forward and backward loop counts. Schwethelm et al. (2026) compare full BPTT with truncation to half the loops, fitting a separate scaling law to each recipe. Parcae (Prairie et al., 2026) varies backward-loop counts in small-model experiments but fixes them to half the forward-loop count in its scaling study, leaving their joint optimization to future work. These studies characterize specific recipes but do not provide a unified law for choosing both loop counts across model sizes and token budgets (Table 1). We thus ask:

How do model size, training tokens, and forward and backward loops jointly affect loss, and should we truncate backpropagation under a fixed training compute budget?

To characterize how these quantities affect loss, we conduct nearly 600 training runs, including a scaling suite spanning 58M–919M non-embedding parameters, over three orders of magnitude in training tokens per parameter, and up to 16 forward loops. We independently vary forward and backward loop counts across model sizes and token budgets and fit a unified scaling law. The law expresses the benefit of looping through an *effective parameter count*—the size of a non-looped model that achieves the same loss at the same token budget. Fully trained loops increase the effective parameter count with diminishing returns, while truncation discounts this gain.

We then use the law to decide whether to truncate, and the answer depends on whether the forward-loop count is prescribed. When it is, the law recovers the benefit of truncation reported in prior work (Schwethelm et al., 2026): the saved backward compute supports a larger model and more training tokens, which more than offset the discounted loop gain. When both loop counts can be chosen, however, the law favors full BPTT. Reducing the forward-loop count frees compute for additional backward loops until every remaining loop is trained, and under the fitted law this reallocation yields more effective parameters at the same compute than the original truncated setting. Separately fitted laws and matched-compute experiments support this advantage (§5).

Building on this preference for full BPTT, we additionally study how to allocate compute among loop count, training tokens, and model size. At fixed model size, additional loops offer diminishing gains while increasing the cost per token, so the law allocates most additional compute to training tokens and increases the loop count only slowly. When model size is also free, compute can instead support more unique parameters; under our leading-FLOP approximation, the law favors a non-looped model and identifies two fully trained loops as the best looped setting. In experiments, models with two fully trained loops nearly match compute-optimal non-looped models at equal training compute while having substantially fewer non-embedding parameters. With a little more training compute, such a smaller model surpasses the compute-optimal non-looped model (§6).

Our contributions are threefold:

- **A unified scaling law based on effective parameters.** We develop and validate a law that expresses loop gains and truncation discounts through an effective parameter count, jointly modeling model size, training tokens, and independent forward and backward loop counts (§4).
- **An explanation of when truncation pays.** We show why truncation can pay at a prescribed forward-loop count and why joint forward–backward allocation favors full BPTT under the fitted law, supported by matched-compute experiments (§5).
- **Compute-allocation rules and parameter-efficient training.** We derive allocation rules for fixed and free model size and demonstrate that two fully trained loops nearly match compute-optimal non-looped models while having substantially fewer trainable parameters (§6).

Figure 1 previews the three results: loops add effective parameters and truncation discounts them; at fixed model size the best loop count grows with compute; two fully trained loops match the loss of a non-looped model with fewer parameters for a little more compute.

2 Related work

Looped transformers. Depth-wise parameter sharing in Universal Transformers and ALBERT decouples computational depth from the number of trainable parameters (Dehghani et al., 2019; Lan et al., 2019). Theoretical and controlled studies show how recurrence supports iterative programs, in-context learning algorithms, and depth-sensitive reasoning (Giannou et al., 2023; Saunshi et al., 2025; Yang et al., 2024). In language model pretraining, Huginn uses a prelude–recurrent–coda architecture (Geiping et al., 2025), while Ouro scales looped pretraining to 7.7T tokens (Zhu et al., 2025). Other work introduces recurrence into pretrained models (Bae et al., 2025a; McLeish et al., 2025) or adapts depth per token (Bae et al., 2025b). Recent architectures improve recurrence through residual and hyper-connections (Fu et al., 2026; Marchenko et al., 2026; Prairie et al., 2026; Zeitoun et al., 2026) or combine it with MoE routing (Chen et al., 2026a; Csordás et al., 2024; Gao et al., 2026; He et al., 2026). These studies show that looping can improve parameter efficiency across architectures; we instead fix the architecture and ask how to train its loops and allocate compute.

Training through recurrent depth. Truncated BPTT limits gradient propagation to a subset of recurrent steps, saving activation memory and backward compute at the cost of biased gradients (Tallec and Ollivier, 2017; Williams and Peng, 1990). Looped language models often supervise the final output and backpropagate through only the last few loops: Huginn and retrofitted models use at most eight backward steps (Geiping et al., 2025; McLeish et al., 2025). Hierarchical recurrent models likewise use a short backward horizon (Wang et al., 2025, 2026a). Other approaches use implicit differentiation (Bai et al., 2019), fixed-point gradient approximations (Huang et al., 2026), or supervision within recurrent segments (He et al., 2026). In these recipes the backward depth is fixed; we vary it independently of the forward depth, weighing the compute that truncation saves against the loop gain it costs.

Scaling laws for recurrence. Size–data scaling laws quantify how parameters and training tokens jointly determine loss (Hoffmann et al., 2022; Kaplan et al., 2020; Videau et al., 2026), and their compute-optimal predictions depend on how the laws are fitted and compute is counted (Besiroglu et al., 2024; Choshen et al., 2025; Li et al., 2025a; Pearce and Song, 2024; Porian et al., 2024). Extensions account for architectural choices such as MoE sparsity (Abnar et al., 2025; Clark et al., 2022; Krajewski et al., 2024). For looped models, some laws fit each architecture separately (Chen et al., 2026c; Lee et al., 2026; Wang et al., 2026b), while others model the loop count explicitly

Table 1 | Scaling laws for looped language models. Earlier laws either fit a separate curve per architecture or model looping through a single loop count T, with backward loops B fixed or tied to T; ours models T and B independently with a bounded effective-parameter gain. /: not discussed; †appendix scaling fit.

		Scaling axes						
Study	Arch.	N	D	T	B	Loop gain	BPTT	Regime
Schwethelm et al. (2026)	dense	✓	✓	✓	✗	power law (T^{ϕ})	T; $\lceil T/2 \rceil$	fixed depth
Parcae (Prairie et al., 2026)	dense	✓	✓	✓	✗	linear (T)	$\lceil T/2 \rceil$	fixed N
Chen et al. (2026b)	dense, MoE	✓	✓	✓	✗	bounded exp. (T)	full (T)	free N, mem.
SMELT (Wang et al., 2026b)	MoE	✓	✓	✗	✗	per-arch. fit	/	matched N, KV
Lee et al. (2026)	dense, MoE	✓	✗	✗	✗	per-arch. fit	/	free N
Chen et al. (2026c)	dense	✓	✓	✗	✗	per-arch. fit	full (T)	free N
Ouro† (Zhu et al., 2025)	dense	✓	✓	✓	✗	additive (T)	/	/
Ours	dense	✓	✓	✓	✓	bounded (T, B)	$1 \le B \le T$	fixed & free N

(Table 1) through additive loss terms (Zhu et al., 2025) or effective parameters with linear, power-law, or bounded gains (Chen et al., 2026b; Prairie et al., 2026; Schwethelm et al., 2026). Among these scaling-law studies of looped transformers, Iso-Depth (Schwethelm et al., 2026) and Parcae (Prairie et al., 2026) are particularly related to our work. Iso-Depth fits separate laws for full and half-length backpropagation and finds that truncation weakens the loop gain but lowers the loss at a fixed forward-loop count by freeing compute. At fixed model size, Parcae finds that loops and tokens should grow together with compute, but ties the backward loops to half the forward loops and leaves their joint choice open. Our law instead treats the backward-loop count as a separate variable, combining coupled size–data scaling (Videau et al., 2026) with bounded loop gains (Chen et al., 2026b). It recovers the truncation benefit at a prescribed forward-loop count but favors full BPTT when both counts are free (§5). In looped MoE models, two loops are the common choice (He et al., 2026): SMELT finds them better than three or four under matched budgets (Wang et al., 2026b), and Loopie finds the marginal benefit of recurrence largest at two (Gao et al., 2026). For dense models, our law ranks two fully trained loops first among looped settings, since they provide nearly the effective size of two untied copies of the block while further loops add less (§6.2). In our runs, two-loop models nearly match compute-optimal models without looping at equal compute while having substantially fewer non-embedding parameters (§6.3).

3 Preliminaries

Looped transformers. We study the prelude–recurrent–coda architecture (Geiping et al., 2025; Schwethelm et al., 2026) (§A.2), which places a shared recurrent block $\mathcal{R}$ (ℓ_r layers) between a prelude $\mathcal{P}$ (ℓ_p layers, including the input embedding) and a coda $\mathcal{C}$ (ℓ_c layers, including the output head); see Figure 1a and §A (Figure 12). While $\mathcal{P}$ and $\mathcal{C}$ run once per token, $\mathcal{R}$ runs for T recurrent iterations (*loops*); for an input x and loops $t = 1, \dots, T$, the model computes

$$s_0 = \mathcal{P}(x), \qquad s_t = \mathrm{RMSNorm}(\mathcal{R}(W_{\mathrm{inj}}\,[\,s_0\,;\,s_{t-1}\,])), \qquad p = \mathcal{C}(s_T). \tag{3.1}$$

(prelude; loop block; input injection; coda)

Here all hidden states have width d, $[\,s_0\,;\,s_{t-1}\,]$ concatenates the prelude output with the previous state, $W_{\mathrm{inj}} \in \mathbb{R}^{d \times 2d}$ projects it back to width d, and the cross-entropy loss applies only to the final output distribution p. We split the non-embedding parameters N into $N = N_{\mathrm{once}} + N_{\mathrm{rec}}$, where N_{once} counts $\mathcal{P}$ and $\mathcal{C}$ (run once per token) and N_{rec} counts $\mathcal{R}$ and W_{inj} (reused each loop; Table 3).

Loop settings, full BPTT, and truncated BPTT. A *loop setting* (T, B) with $1 \le B \le T$ executes T forward loops and backpropagates gradients through only the last B loops (Figure 1a). During

training, the state s_{T-B} is detached from the computation graph, so the first $T-B$ loops run without gradients; because s_0 is injected at every loop in (3.1), the prelude $\mathcal{P}$ retains a direct gradient path for any $B \geq 1$. Thus $B = T$ gives full BPTT, $B < T$ gives truncated BPTT, and $(1, 1)$ recovers a standard non-looped transformer.

Baseline scaling law. To relate validation loss to non-embedding parameter count N and training tokens D for a non-looped model, we adopt the coupled scaling law of Videau et al. (2026),

$$L(N, D) \;=\; E + \Big[A\Big(\frac{N}{N_0}\Big)^{-\alpha} + H\Big(\frac{D}{D_0}\Big)^{-\beta}\Big]^{\kappa}, \tag{3.2}$$

where E is the irreducible loss, N_0 and D_0 are fixed reference scales, $\alpha, \beta > 0$ are the size and token exponents, and the outer exponent κ couples the two terms ($\kappa = 1$ recovers the additive form of Hoffmann et al. (2022)); we denote the data coefficient by H to reserve B for backward loops. In §4, we extend (3.2) to looped models by replacing N with an effective parameter count in which (T, B) rescales N_{rec} by a *looping coefficient* $g(T, B)$.

4 A scaling law for forward and backward loops

Should a looped transformer truncate backpropagation? At a fixed model size and training compute, truncation saves backward computation that can be spent on more forward loops or more training data, but it also reduces the benefit of looping. We fit a scaling law that separates forward and backward loop counts to quantify this tradeoff. This section fits and validates the law and interprets it in terms of effective parameters. §5 then uses the law to decide whether to truncate, and §6 uses it to divide compute among loops, data, and model size.

4.1 Fitting and validating the scaling law

Experimental setup. We follow the architecture and training recipe of Schwethelm et al. (2026), which builds on the prelude–recurrent–coda layout of Huginn (Geiping et al., 2025) (§3), while holding the unique non-embedding parameters N fixed across loop settings as in Prairie et al. (2026). We train five model sizes from 58M to 919M non-embedding parameters (Table 3) on the 100B-token FineWeb-Edu corpus (Penedo et al., 2024) with MuonH and AdamW (Jordan et al., 2024; Wen et al., 2025), expressing token budgets D in units of $D_0 = 2^{30} \approx 1.07 \times 10^9$ tokens and reporting validation cross-entropy in nats per token on deduplicated held-out shards. The results in this paper come from 592 training runs, described with the full setup in §A.2. Of these, 385 form the *scaling suite*: 343 configurations of the five model sizes from 1.1 to 1,291 tokens per parameter, in fifteen loop settings (T, B) spanning $T \in \{1, 2, 4, 8, 16\}$ and $1 \leq B \leq T$ (§F); because a backward loop costs twice a forward loop, loop-dependent training compute scales with $T + 2B$ (§C.1). We fit the law on the 226 of them on four diagonals of nearly constant tokens per parameter (about 9, 18, 37 and 74), plus four configurations from two compute-matched pairs, for 230 configurations in total (§B.3). The other 117, mostly at shorter or longer budgets, serve as separate tests.

Effective-parameter scaling law. Only the recurrent block runs in every loop, so looping can enlarge only this block. We therefore write the loss as the coupled law of Videau et al. (2026) with an effective parameter count. In it, the recurrent block, N_{rec}, counts several times, and the prelude and coda, N_{once}, count once (§3):

$$\begin{aligned} L &= E + \Big[A\Big(\frac{N_{\text{eff}}}{N_0}\Big)^{-\alpha} + H\Big(\frac{D}{D_0}\Big)^{-\beta}\Big]^{\kappa}, \\ N_{\text{eff}} &= N_{\text{once}} + g(T, B) N_{\text{rec}}, \qquad g(T, B) = \exp\big[c\,(1 - T^{-p} B^{-q})\big]. \end{aligned} \tag{4.1}$$

Here $g(T, B)$ is the *looping coefficient*: it multiplies the recurrent block's parameter count, while the prelude and coda count once. With full BPTT, the looping coefficient $g(T, T) = \exp[c(1 - T^{-(p+q)})]$ grows with T; truncation multiplies it by $g(T, B)/g(T, T) \leq 1$.

Table 2 | Fitted parameters on the 230-configuration fitting set. The reference scales are fixed at $N_0 = 2.4 \times 10^8$ parameters and $D_0 = 2^{30}$ tokens.

E	A	H	α	β	κ	c	p	q
0.29865	6.48695	7.63343	0.30568	0.39281	0.32510	1.11198	0.23677	1.02181

Design of the looping coefficient. We choose $g(T, B)$ to satisfy four modeling requirements:

1 Normalization

$$g(T, B) \geq 0, \qquad g(1, 1) = 1$$

The looping coefficient is nonnegative; a single loop recovers the non-looped model.

2 Monotonicity

$$\frac{\partial g}{\partial T} > 0, \qquad \frac{\partial g}{\partial B} > 0$$

Increasing either loop count increases the looping coefficient.

3 Zero-backward boundary

$$g(T, 0) = 0$$

Untrained recurrent block contributes no effective size.

4 Saturation

$$\sup_{1 \leq B \leq T} g(T, B) < \infty$$

Finite parameters store finite information, motivating a bounded gain.

The finite-ceiling assumption is supported by prior observations of saturation: Chen et al. (2026b) model a bounded effective-parameter gain from looping, while Parcae finds that adding test-time loops to a trained model approaches a loss floor (Prairie et al., 2026). There is also an information-storage motivation. At a fixed parameter count and weight precision, the weights can store only a finite amount of information; additional loops provide more computation over those weights without increasing their storage capacity. We therefore treat saturation as an empirical modeling assumption and learn its ceiling from the data.

These requirements motivate the form in (4.1). Nonnegativity motivates an exponential parameterization, ensuring $g(T, B) > 0$ for $T, B > 0$. With $c, p, q > 0$, increasing either loop count decreases $T^{-p} B^{-q}$, so $g(T, B)$ grows toward the finite ceiling e^c. The shared coefficient c enforces $g(1, 1) = 1$, and continuous extension as $B \to 0^+$ gives $g(T, 0) = 0$. At fixed model size and token budget, p governs the loss reduction from extra forward loops at fixed B, while q governs the loss reduction from extra backward loops at fixed T. These three loop coefficients, c, p, q, together with the six coefficients $E, A, H, \alpha, \beta, \kappa$ of the coupled size–data law (3.2), give nine fitted coefficients.

Empirical looping coefficients. Figure 2 motivates coupling T and B: the empirical looping coefficient has a smaller fitted log–log slope against $r = B/T$ at larger T. Figure 3 compares the shared looping-coefficient surface with empirical looping coefficients. For each looped run with loss ℓ, we first use shape-preserving piecewise cubic Hermite interpolation (PCHIP) (Fritsch and Butland, 1984) to interpolate the non-looped measurements at the same target token budget to obtain the effective size $\widehat{N}_{\text{eff}}$ matching ℓ. We then use the effective-size relation in (4.1): subtracting N_{once} and

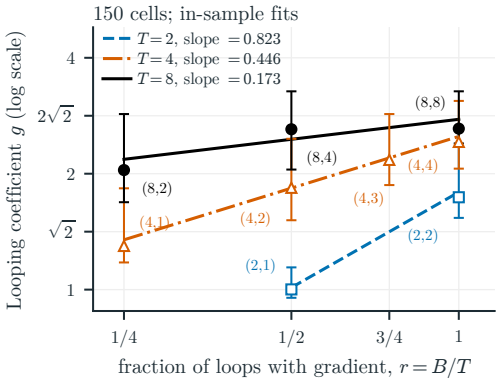


Figure 2 | The looping coefficient varies with $r = B/T$. Markers and bars: medians and ranges of 150 loss-matched empirical coefficients. Lines: in-sample loss-matched $\log_2 \hat{g}$ against r (§B.2).

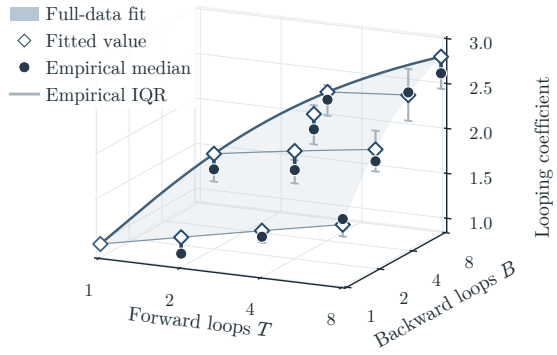


Figure 3 | The looping coefficient and the runs. Surface and diamonds: the full-data fitted looping coefficient $g(T, B)$. Circles and gray bars: medians and interquartile ranges (IQR) of loss-matched empirical looping coefficients across 163 configurations with $T \leq 8$ (§B.2).

dividing by N_{rec} gives the empirical looping coefficient $\hat{g}$ (§B.2). Vertical segments join each empirical median to the fitted coefficient at the same loop setting. This agreement shows that $g(T, B)$ with just three parameters c, p, q captures the observed loop dependence; Table 4 quantifies the law's in-sample fit and leave-one-out (LOO) prediction errors.

Fitting procedure. We fit all nine parameters jointly by minimizing the sum of squared differences between predicted and measured validation losses, giving each configuration equal weight. The reference scales $N_0 = 2.4 \times 10^8$ parameters and $D_0 = 2^{30}$ tokens are held fixed. Predictions use actual token counts D; budget holdouts group target token budgets. To reduce sensitivity to the initial guess, we fit the law 64 times from different random starting values and retain the solution with the smallest fitting error. All fits converge. Table 2 gives the parameters fitted on the 230 configurations used for the validation below. Further fitting details are in §B.

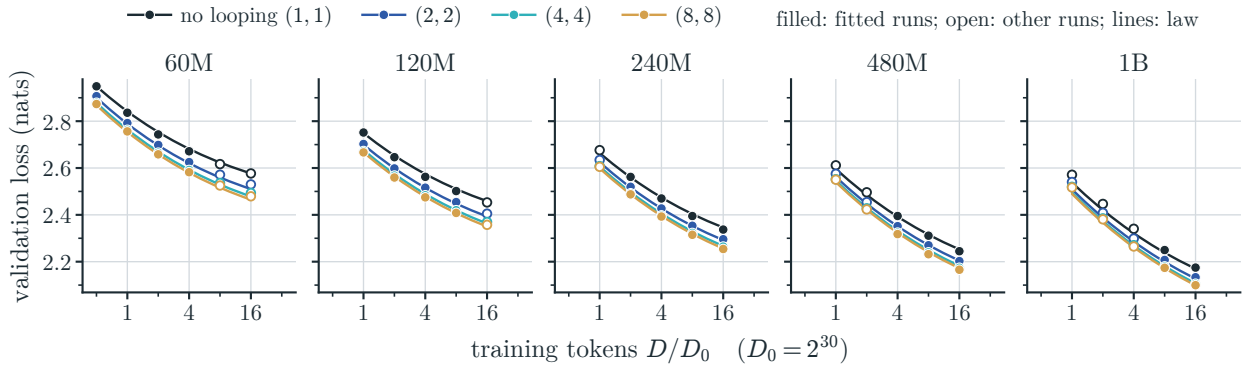


Figure 4 | More loops with full BPTT lower the loss at every size and budget and one law tracks all of them. Validation loss against training tokens for the four loop settings with full BPTT. Dots: runs, open where outside the fitting set; lines: the full-data fitted law (§4).

Holdout validation. To test whether the law predicts measurements it has not been fitted to, we leave out selected configurations, fit all nine parameters again using the remaining measurements, and compare the predictions with the omitted measurements. For example, to test prediction at an unseen model size, we exclude every configuration at that size and fit the law using the other

sizes. We repeat this for each size and for the other exclusions listed in Table 4.

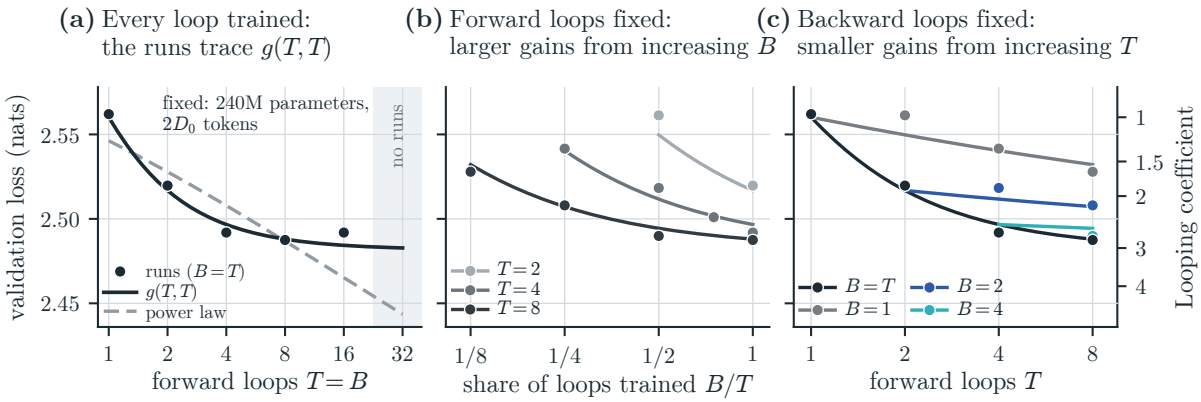


Figure 5 | Forward and backward loops at fixed size and budget (240M, $2D_0$). Markers show measured runs and solid lines show the fitted law. **(a)** $B = T$: the gain levels off; dashed, the power-law gain of prior work (Schwethelm et al., 2026) keeps growing. **(b)** Fixed T: training more of the loops lowers the loss. **(c)** Fixed B: more forward loops help little.

Prediction accuracy. The law tracks the measured loss curves across sizes and token budgets (Figure 4), including the held-out configurations (marked with open circles). Figure 5 gives the complementary view across loop settings at a fixed model size and token budget: varying T with $B = T$, varying B at fixed T, and varying T at fixed B. Both figures use the law fitted on the 230-configuration fitting set; the curves are evaluated on both fitted and held-out configurations. §F overlays the law on the whole scaling suite.

Holding out a model size or token budget increases prediction error only modestly (Table 4), whereas unseen loop settings are harder to predict (§B). Extrapolation back to the non-looped baseline is the weakest loop-count check; §B reports the boundary errors and alternative gains. The table separates fit quality from held-out prediction error. §4.2 examines the remaining discrepancies in effective-parameter units (Figure 6), and §6.1 checks their effect on compute allocation (Figure 10).

Table 3 | Model sizes.

Size	d	Layers	N	N_{rec}/N
60M	768	2/4/2	57.8M	0.510
120M	1024	2/5/2	115.4M	0.564
240M	1280	3/6/3	239.2M	0.507
480M	1664	3/7/4	470.8M	0.506
1B	2176	4/8/4	918.7M	0.505

d: width; layers: $\ell_p/\ell_r/\ell_c$; N: non-embedding parameters (§A.2).

Table 4 | Fit and prediction errors ($\times 10^{-3}$).

Evaluation	Fits	RMSE	MAE
In-sample	1	6.14	4.91
One configuration out	230	6.40	5.11
One model size N out	7	7.48	5.89
One token budget D out	11	6.42	5.12
One (T, B) setting out	15	8.67	6.74

Errors in $\times 10^{-3}$ nats/token over 230 configurations ($R^2 = 0.9992$; §B).

4.2 Illustrating the scaling law with effective parameters

How many parameters is a loop setting worth? (4.1) allows us to decompose the answer into the gain from fully trained loops, $g(T,T)$, and the discount for truncation, $g(T,B)/g(T,T)$. We use the empirical looping coefficients introduced above to compare the law with the runs in Figure 6; their construction is detailed in §B.2.

The looping coefficient saturates for large T. With full BPTT, $B = T$, and the gain is $g(T,T) = \exp[c(1 - T^{-(p+q)})]$. The runs in Figure 5a vary both loop counts together at fixed model size and

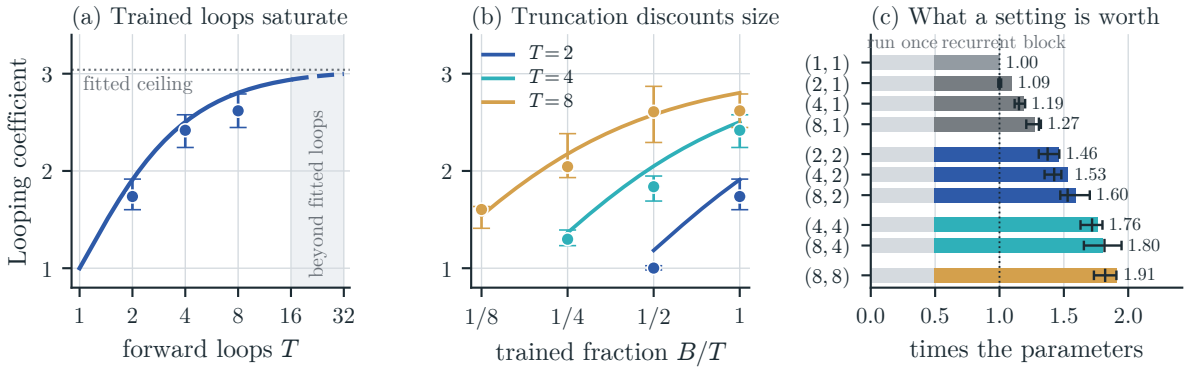


Figure 6 | **What a loop setting is worth in effective size. (a)** Full BPTT, the looping coefficient $g(T, T)$, approaches its fitted ceiling (dotted); dashed beyond the fitted loop counts. **(b)** The looping coefficient $g(T, B)$ against the trained fraction B/T. **(c)** Total effective size N_{eff}/N, grouped by B. Lines, bars and numbers: the law; markers and error bars: run estimates, median and interquartile range over sizes and budgets (§B.2).

token budget. Each doubling of trained loops adds less than the one before (Figures 5a and 6a), as Chen et al. (2026b) also find with all loops trained. The power-law gain of prior work (Schwethelm et al., 2026) keeps growing, whereas ours approaches e^c. Our runs support the bounded form over the power law, predicting held-out forward-loop counts more accurately (§B.4).

Doubling B outweighs doubling T alone. At fixed T, truncation reduces B and hence $g(T, B)$, consistent with the smaller effective sizes in Figure 6b and higher losses in Figure 5b. Figure 6c clarifies the relative roles of T and B by grouping settings with the same B. The fitted bars and empirical estimates change relatively little as T increases within a group, but show larger gains as B increases across groups, including comparisons at the same T. The fitted exponents quantify this asymmetry: $q > p$ (Table 2) implies $g(T, 2B) > g(2T, B)$ for $2B \leq T$. From the same initial setting at fixed model size, doubling B thus adds more effective parameters than doubling T alone; the small loss gains from extra forward loops at fixed B support the same conclusion (Figure 5c).

> **Finding 1** **Loops add effective parameters; truncation discounts them.** Each fully trained loop adds effective parameters, with diminishing returns toward a finite ceiling. Truncation discounts this gain, and from the same setting, doubling B adds more effective parameters than doubling T.

5 Truncate or not?

Truncation saves backward compute but reduces the effective parameter count (§4.2). Whether it lowers loss at a given compute depends on which quantities are held fixed. We first fix the forward-loop count, as prior scaling studies do, and then let both loop counts vary at equal compute.

5.1 Fixed forward loops: truncation can be optimal

Fix T and optimize N, D, B at compute C with fixed parameter proportions. The effective size is $N_{\text{eff}} = N_{\text{once}} + g(T, B)N_{\text{rec}}$, and the *compute-equivalent size* N_{comp} sets the leading cost:

$$C \simeq 6DN_{\text{comp}}, \qquad N_{\text{comp}} = N_{\text{once}} + \frac{T + 2B}{3} N_{\text{rec}}. \tag{5.1}$$

Table 5 | Predicted optimal allocations at fixed T, with $N_{\text{rec}}/N \simeq 0.507$. We optimize N, D under leading FLOPs and enumerate all integer $1 \le B \le T$. Each (N^*, D^*) pair is normalized by the respective full-BPTT optima at the same T, C; these ratios are independent of C.

T	B^*	$\chi(T, B^*)$	$\chi(T, T)$	N^*	D^*
1	1	1.000	1.000	1	1
2	2	1.031	1.031	1	1
4	2	1.204	1.430	1.27	1.08
8	2	1.580	2.376	1.51	1.20
16	2	2.343	4.339	1.70	1.31

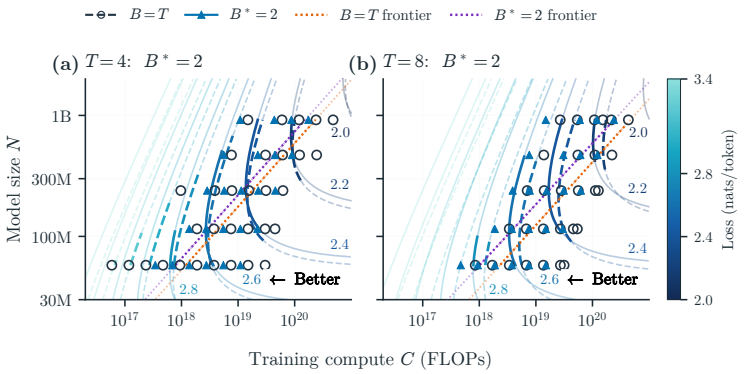


Figure 7 | **Separately fitted laws confirm the predicted truncation.** Iso-loss contours (labels: loss in nats) and compute-optimal model sizes of laws fitted separately to $B = T$ and to the predicted $B^* = 2$. Markers are measured runs.

This counts two FLOPs per parameter per forward pass and four per backward pass, omitting attention products, the output projection, and optimizer updates. Under this approximation, Lemma C.1 in §C.4 shows that optimizing the law over unrestricted positive, continuous N, D reduces to

$$B^* \in \operatorname*{arg\,min}_{B \in \{1,\dots,T\}} \chi(T, B), \qquad \text{where} \quad \chi(T, B) = \frac{N_{\text{comp}}}{N_{\text{eff}}}.$$

Smaller χ means less compute per effective parameter; its ranking is independent of C under these assumptions. The stationary condition (C.7) gives continuous candidates for B^*; integer choices are evaluated directly using χ.

Truncation reduces effective size, but frees compute for a larger model and more tokens. At $T = 8$, the law selects $B^* = 2$ and predicts a 34% compute saving at equal optimized loss (Table 5). Separately fitted laws at $T = 4, 8$ also favor $B = 2$ over $B = T$: at $T = 8$, their compute-optimal frontiers predict a 33–35% saving within the range of the runs (Figure 7; fitting details in §C.4).

This agrees with the fixed-compute truncation benefit reported by Schwethelm et al. (2026). Parcae uses truncated BPTT but leaves the compute-optimal choice of forward and backward loop counts to future work (Prairie et al., 2026, Appendix I). The benefit is conditional on prescribing T. We next ask whether it survives when both loop counts can be chosen.

5.2 Free forward-loop count: full BPTT is optimal

When T can also be chosen, we optimize the split between forward and backward loops at equal compute. Fix the model architecture and token count D. In our FLOP accounting, each forward pass through the recurrent block costs one unit and each backward pass costs two, including their attention operations (§C.1). The recurrent block thus costs $T + 2B$ units per token; all other model components have the same cost across loop settings. Holding $T + 2B$ fixed therefore holds total compute fixed: for example, $(T, B) = (4, 4)$ and $(8, 2)$ both cost 12 recurrent units per token.

Under our scaling law, for a loop-compute count $S \ge 3$, minimizing loss at this compute is equivalent to maximizing the looping coefficient $g(T, B)$:

$$\operatorname*{maximize}_{T, B \in \mathbb{R}} \quad g(T, B) \qquad \text{subject to} \quad T + 2B = S, \quad 1 \le B \le T. \tag{5.2}$$

Table 6 | Reducing forward depth at equal training compute. Measured validation losses at $D = 16D_0$ (17.2 billion tokens). Within each row, $(T, B) = (8, 2)$ and $(4, 4)$ have identical model size, tokens, and training compute C, including optimizer updates. Losses and the reduction $\Delta L = L_{8,2} - L_{4,4}$ are in nats per token. Model sizes use nominal labels; C is the training compute in FLOPs.

Size	C (10^{19})	$L_{8,2}$	$L_{4,4}$	ΔL (10^{-2})
60M	2.41	2.5211	2.4933	2.78
120M	4.59	2.3843	2.3696	1.47
240M	8.28	2.2778	2.2642	1.36
480M	15.29	2.1845	2.1738	1.07
1B	28.38	2.1144	2.1057	0.87

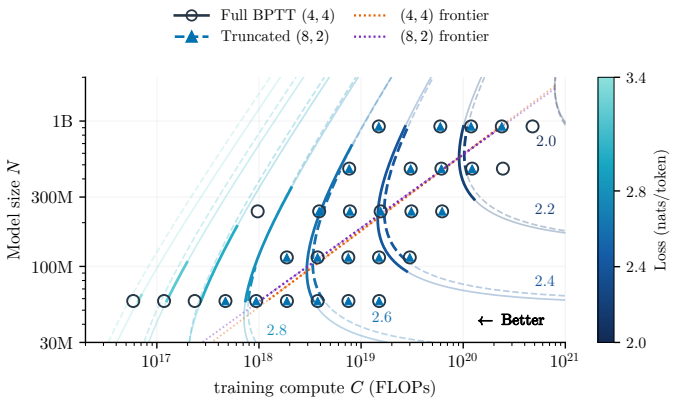


Figure 8 | Fewer fully trained loops improve the compute-optimal frontier. Iso-loss contours of separately fitted laws for $(T, B) = (4, 4)$ (solid) and $(8, 2)$ (dashed), both with $T + 2B = 12$. At equal loss, solid contours lie left of dashed ones: $(4, 4)$ uses 8–14% less compute.

The optimal split uses full BPTT. Recall that $g(T, B) = \exp[c(1 - T^{-p}B^{-q})]$ with $c > 0$. Thus (5.2) is equivalent to minimizing $T^{-p}B^{-q}$ under the same affine constraints. This is a standard convex optimization problem: whenever $q > 2p$, its unique solution is $T = B = S/3$ for every $S \geq 3$, giving full BPTT (Lemma C.2 in §C.4.2). The fitted exponents in Table 2, $p \simeq 0.237$ and $q \simeq 1.022$, satisfy $q > 2p \simeq 0.474$ by a wide margin. Therefore, the law predicts that full BPTT is optimal at every compute if both T and B can be chosen continuously.[1]

Separately fitted laws confirm the benefit of full BPTT. The compute-optimal frontiers of the separately fitted laws predict that $(4, 4)$ reaches the same loss with less compute than $(8, 2)$ where both optimal allocations lie within the ranges of their runs (Figure 8). Matched-compute runs agree: at $D = 16D_0$, $(4, 4)$ achieves lower loss than $(8, 2)$ at every size (Table 6).

> **Finding 2** **Truncate only when the forward-loop count is prescribed.** When T is prescribed, the law favors truncation, and separately fitted laws agree. When both loop counts can be chosen, full BPTT is optimal at every size and compute, since one backward loop is worth more than the two forward loops of the same cost ($q > 2p$).

6 Allocating compute among loops, data, and model size

Given full BPTT, it remains to divide training compute among loops, training tokens, and, when it is not fixed, model size. Following Hoffmann et al. (2022), we summarize each allocation by how its optimum scales with compute, $X^* \propto C^a$. We first fix the model size (§6.1) and then let it vary (Sections 6.2 and 6.3).

[1] The fully trained replacement may not be an integer setting: $(4, 3)$ would become $(10/3, 10/3)$. On our integer menu, the law gives $(4, 3)$ a lead over both $(2, 2)$ and $(4, 4)$ of at most 0.002 nats over a narrow compute range, while in the runs the best fully trained setting beats the best truncated one at every size and compute (Figure 10).

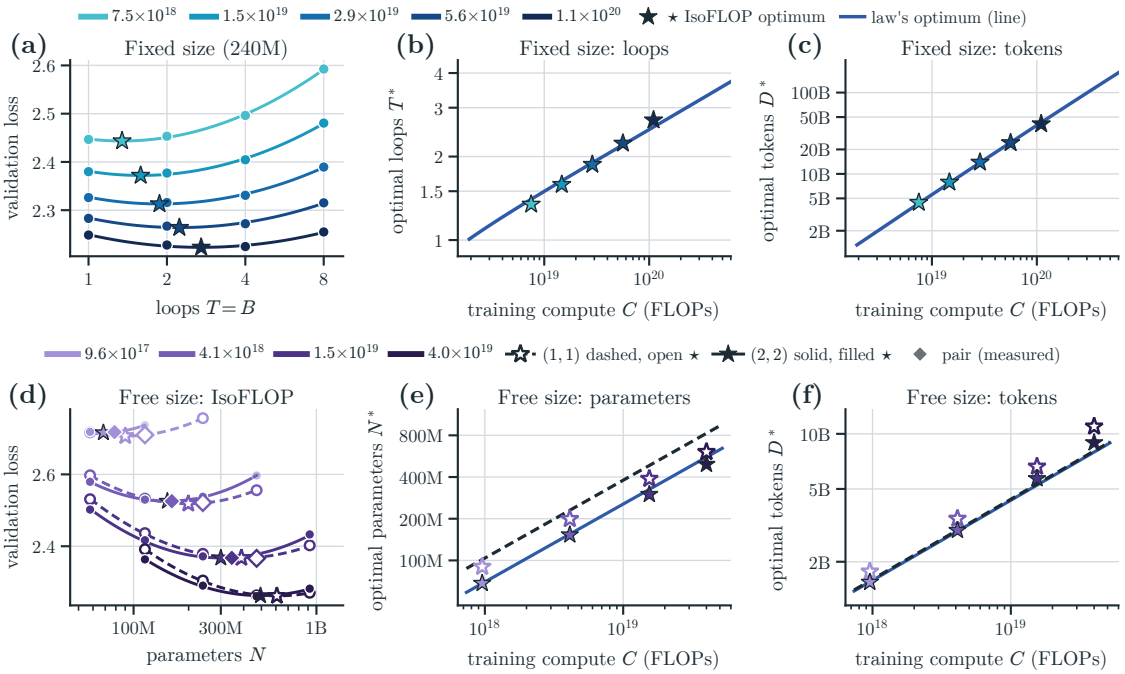


Figure 9 | **How the compute-optimal allocation scales with training compute. Top:** fixed model size (240M), our law predicts an optimal allocation between T^* and D^*. (a) The loss at five computes against the loop count. Its vertex (star) is the optimum at that compute. (b, c) The optimal loop count T^* and token count D^* of the stars, and the law's optimum (line), whose exponents approach (6.4) asymptotically. **Bottom:** free model size, $(T, B) = (2, 2)$ tracks $(T, B) = (1, 1)$ closely in IsoFLOP fitting. (d) The loss at four computes against model size, with its minimum (star); diamonds: the compute-optimal pairs of §6.3. (e, f) N^* and D^* of the stars and the law's optimum (6.5), an exact power law and so a straight line; the law places the optimum of $(1, 1)$ at a larger model than its runs do, and its two lines nearly coincide in (f). Colours mark compute within each row. Compute is $6N_{\text{comp}}D$ (5.1) (under which the three pairs of Table 11 lie at 9.6×10^{17}, 4.1×10^{18}, and 1.5×10^{19} FLOPs).

6.1 Fixed model size: loops versus data

Suppose a model must fit within a device's memory budget, so adding parameters is not an option. With more training compute available, should we reuse those parameters through more loops per token, or train on more tokens? At fixed size, more loops raise effective size but leave fewer training tokens within the same compute budget; our law balances these two.

Asymptotic loop–token allocation under full BPTT. The allocation power laws below describe the large-compute limit of our fitted loss law, not a separate fit to measured optima. Fix the architecture and parameter counts ($N_{\text{rec}} > 0$); setting $B = T$ in (4.1) gives the full-BPTT loss

$$L_N(T, D) = E + \left[A \left(\frac{N_{\text{once}} + N_{\text{rec}} e^{c(1-T^{-(p+q)})}}{N_0} \right)^{-\alpha} + H \left(\frac{D}{D_0} \right)^{-\beta} \right]^{\kappa}. \tag{6.1}$$

Because $\kappa > 0$, minimizing (6.1) is equivalent to minimizing its inner sum, $J_N = (L_N - E)^{1/\kappa}$. As $T \to \infty$, the effective size approaches $N_\infty = N_{\text{once}} + e^c N_{\text{rec}}$. Expanding around this limit gives

$$J_N(T, D) = J_\infty + K_N T^{-(p+q)} + H \left(\frac{D}{D_0} \right)^{-\beta} + O\left(T^{-2(p+q)}\right), \tag{6.2}$$

where $J_\infty = A(N_\infty/N_0)^{-\alpha}$ and $K_N = \alpha c(N_{\text{rec}}e^c/N_\infty)J_\infty > 0$ are fixed by the architecture and fitted coefficients; the leading loop term has exponent $p+q$ and the data term has exponent β.

Following the power-law allocation argument of Hoffmann et al. (2022), we balance these two terms under a compute constraint. At fixed size, compute per token grows linearly with T at large depth ($C \propto TD$). For continuous T, D, minimizing the leading terms in (6.2) gives the asymptotic scaling

$$T^* \propto C^{\frac{\beta}{\beta+p+q}}, \qquad D^* \propto C^{\frac{p+q}{\beta+p+q}} \quad \text{at fixed } N. \tag{6.3}$$

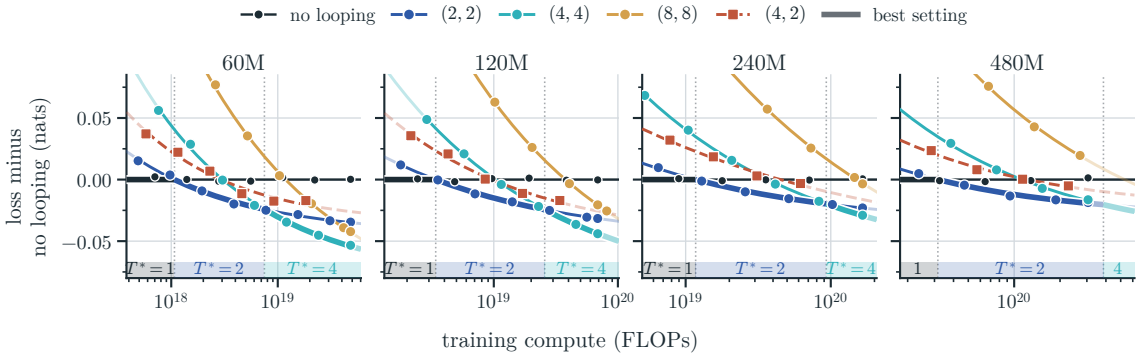


Figure 10 | **At a fixed model size, the best loop count grows with training compute: no looping, then two, then four trained loops.** Loss minus that of the model without looping at the same compute, which spends it on more tokens; markers are runs, with a smooth curve through each setting; $(4, 2)$ trains half of its four loops. The bold curve is the best setting, named in the strip by its loop count T^* (faded where extrapolated). Within each stretch the best model spends added compute on tokens, and at each switch it trades tokens for loops.

These powers need not describe finite-compute optima, where overheads and integer loop counts remain consequential. §C.4.3 derives this limit from the exact stationary condition, including the compute overhead of the non-recurrent layers.

Loops grow slowly; most added compute goes to data. Substituting the fitted $p+q \simeq 1.259$ and $\beta \simeq 0.393$ into (6.3) gives, with rounded exponents,

$$T^* \propto C^{0.24}, \qquad D^* \propto C^{0.76}. \tag{6.4}$$

A tenfold increase in compute therefore raises the optimal loop count by about 1.7 times and the token count by about 5.8 times. These exponents are asymptotic: because loop counts are integers, the best setting changes in steps. Between steps, added compute goes to tokens; at each step, the model switches to a deeper setting with fewer tokens. The runs show the same trend: at each size, the loop count minimizing loss at equal compute grows with compute as predicted (Figure 9a–c).

Comparison with prior scaling laws. Parcae reports empirical exponents of approximately 0.40 for mean recurrence and 0.78 for training tokens (Prairie et al., 2026, Section 5.2). (6.4) agrees that tokens grow faster than loops, but predicts slower loop growth in the asymptotic regime. Parcae samples recurrence and uses truncated BPTT, so its reported optimum is conditional on that training recipe. Our full-BPTT result suggests that truncation may be suboptimal when both loop counts are free under our law and compute model; whether this also holds for Parcae requires a direct comparison. The law of Schwethelm et al. (2026) instead lets the looping coefficient grow unbounded as T^φ. Our gain saturates, and $p+q$ governs the approach to that ceiling, giving a different asymptotic allocation between loops and tokens.

Finding 3 **At fixed model size, spend additional compute mostly on training tokens.** Under the fitted law, the optimal number of fully trained loops grows asymptotically as $C^{0.24}$ and the number of training tokens as $C^{0.76}$; the runs show the same trend.

6.2 Free model size: the law favors no looping

When model size is free, the compute spent repeating a block can instead train more unique parameters. Start with a T-loop model using full BPTT, for integer $T \geq 2$, and replace its shared recurrent block with T untied copies. The resulting non-looped model has the same compute C and token budget D under (5.1), but greater effective size:

$$N_{\text{once}} + g(T,T)N_{\text{rec}} < N_{\text{once}} + TN_{\text{rec}}.$$

The left side is the effective size of the looped model; the right side is that of the model with T untied copies. The inequality follows from the fitted $g(T,T) < T$ (Figure 11; §C.4.4). The non-looped model therefore has greater effective size and lower predicted loss at every token budget. Figure 11 also shows that $g(2,2)$ is close to 2; we examine this two-loop case in detail in §6.3.

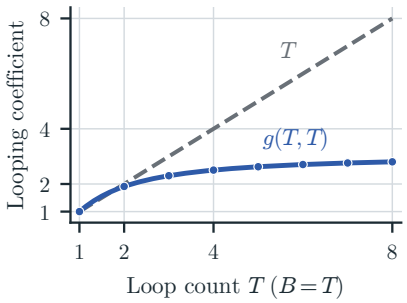


Figure 11 | Fitted looping coefficient $g(T,T)$ versus T untied copies. Markers indicate integer T.

The best loop setting does not change with compute. With parameter proportions fixed and positive, continuous N, D unrestricted, every loop setting shares the compute exponents of the model without looping. Optimizing the loop setting as well gives

$$N^* \propto C^{0.56}, \qquad D^* \propto C^{0.44}, \qquad T^* = B^* = 1. \tag{6.5}$$

The exponents are rounded predictions under the leading-FLOP count (§C.4.4). Under these assumptions, the ranking of loop settings is the same at every compute.

No looping is optimal, and two fully trained loops come closest. Among the fifteen settings, the law favors the model without looping at fixed compute, consistent with Schwethelm et al. (2026). The best looped setting is $(2,2)$ (Table 5): $g(2,2) \approx 2$, so two shared loops provide nearly the effective size of two untied copies. Because $N_{\text{eff}} > N$ for $(2,2)$, the law predicts that it comes close to the model without looping while having fewer trainable parameters. §6.3 tests this prediction.

6.3 Two fully trained loops are competitive with fewer parameters

Near parity at equal compute. At each of three training budgets, we train a model with two fully trained loops, $(2,2)$, and a model without looping, each at the size and token count that minimize its loss for that budget, with three seeds per model (§D). At every budget, the looped model stays within 0.008 nats of the model without looping while having 27–32% fewer non-embedding parameters (Table 11). The gap shrinks as the budget grows, and at 2.1×10^{19} FLOPs[2] it is smaller than its standard error across seeds. Both models sit near the flat minima of their IsoFLOP profiles, the looped one at a smaller size (Figure 9d).

[2]Training compute including the optimizer update, as in Table 11; the forward and backward passes alone take 2.0×10^{19} FLOPs.

Ahead with a little more compute. Because the gap is so small, a little more compute lets the looped model pull ahead. We take the pair at 2.1×10^{19} FLOPs and train its $(2, 2)$ model on 8% more tokens, and so with 8% more compute. Its loss then falls 0.007 nats below that of the model without looping, well beyond the spread across seeds, while it still has 27% fewer non-embedding parameters (Figure 1d; Table 11). Appendix D also compares the looped model with an untied model, which gives each loop its own block, at a fixed number of training tokens.

> **Finding 4** **Two fully trained loops are competitive with fewer parameters.** A model with two fully trained loops nearly matches the compute-optimal model without looping at equal training compute; with 8% more compute, it achieves lower validation loss while having 27% fewer non-embedding parameters.

7 Conclusion

Truncated BPTT saves compute, but whether it lowers the loss depends on which quantities are held fixed. We fit a scaling law that expresses every loop setting as effective parameters, with a gain that saturates and shrinks under truncation (§4). Under this law, truncation helps only when the forward-loop count is prescribed. When both loop counts are chosen, full BPTT is optimal at every model size and compute we analyze (§5).

The law also determines how to allocate compute. At fixed model size, most additional compute should go to data, and the number of fully trained loops grows only as about $C^{0.24}$ (§6.1). When model size is free, the law favors no looping, but two fully trained loops come close: with 27–32% fewer non-embedding parameters they nearly match the compute-optimal non-looped model at equal compute, and with 27% fewer parameters they achieve lower loss at 8% more compute (§6.3).

Limitations. Our main limitation is the focus on dense models: we do not study mixture-of-experts (MoE) architectures, so whether our conclusions on truncation and compute allocation extend to looped MoE models remains open. Second, we measure validation loss only and do not establish whether the observed gains translate to downstream task performance or reasoning ability. Third, our experiments are capped at $T \leq 16$, so the gain ceiling and later switches to more loops remain extrapolations. Related scaling sweeps reach eight loops in Schwethelm et al. (2026) and mean training loop counts of twelve in Parcae (Prairie et al., 2026). Much deeper recurrence can exacerbate training instability, including residual explosion and loss spikes (Fu et al., 2026; Prairie et al., 2026); developing stable training recipes for that regime is beyond our scope. Finally, we fix the ratio of non-embedding parameters in the combined prelude and coda to those in the recurrent block for each model size. Varying this ratio to study how parameter allocation affects the gains from looping and the compute-optimal loop counts is a direction for future work.

Acknowledgments

We thank Yuwei Niu and Yi Ge for valuable feedback and helpful discussions.

References

Abnar, S., Shah, H., Busbridge, D., Ali, A. M. E., Susskind, J. and Thilak, V. (2025). Parameters vs FLOPs: Scaling Laws for Optimal Sparsity for Mixture-of-Experts Language Models. *arXiv preprint arXiv:2501.12370*. 3

Bae, S., Fisch, A., Harutyunyan, H., Ji, Z., Kim, S. and Schuster, T. (2025a). Relaxed Recursive Transformers: Effective Parameter Sharing with Layer-wise LoRA. In *International Conference on Learning Representations (ICLR)*. 3, 34

Bae, S., Kim, Y., Bayat, R., Kim, S., Ha, J., Schuster, T., Fisch, A., Harutyunyan, H., Ji, Z., Courville, A. and Yun, S.-Y. (2025b). Mixture-of-Recursions: Learning Dynamic Recursive Depths for Adaptive Token-Level Computation. *arXiv preprint arXiv:2507.10524*. 3

Bai, S., Kolter, J. Z. and Koltun, V. (2019). Deep Equilibrium Models. In *Advances in Neural Information Processing Systems (NeurIPS)*. 3

Bergsma, S., Dey, N., Gosal, G., Gray, G., Soboleva, D. and Hestness, J. (2025). Power Lines: Scaling Laws for Weight Decay and Batch Size in LLM Pre-training. In *Advances in Neural Information Processing Systems (NeurIPS)*. 22, 36

Besiroglu, T., Erdil, E., Barnett, M. and You, J. (2024). Chinchilla Scaling: A replication attempt. *arXiv preprint arXiv:2404.10102*. 3

Bordelon, B., Noci, L., Li, M. B., Hanin, B. and Pehlevan, C. (2023). Depthwise Hyperparameter Transfer in Residual Networks: Dynamics and Scaling Limit. *arXiv preprint arXiv:2309.16620*. 36

Chen, W., Li, T., Huang, W., Yin, Y., Shang, L. and Qin, C. (2026a). LoopMoE: Unifying Iterative Computation with Mixture-of-Experts for Language Modeling. In *Empirical Methods in Natural Language Processing (EMNLP)*.
https://arxiv.org/abs/2606.04438 3

Chen, Y., Goyal, A. and Krishnamoorthi, R. (2026b). Scaling Laws for Looped Mixture of Experts. *arXiv preprint arXiv:2609.40316*. 4, 6, 9

Chen, Z., Vegesna, A., Dahal, S. and Wilson, A. G. (2026c). How Model Growth, Recursion, and Boundary Operators Influence Scaling Exponents. *arXiv preprint arXiv:2609.19107*.
https://arxiv.org/abs/2609.19107 3, 4

Choshen, L., Zhang, Y. and Andreas, J. (2025). A Hitchhiker's Guide to Scaling Law Estimation. In *International Conference on Machine Learning (ICML)*. 3

Clark, A., de las Casas, D., Guy, A., Mensch, A., Paganini, M., Hoffmann, J., Damoc, B., Hechtman, B., Cai, T., Borgeaud, S., van den Driessche, G., Rutherford, E., Hennigan, T., Johnson, M., Millican, K. et al. (2022). Unified Scaling Laws for Routed Language Models. *arXiv preprint arXiv:2202.01169*. 3

Csordás, R., Irie, K., Schmidhuber, J., Potts, C. and Manning, C. D. (2024). MoEUT: Mixture-of-Experts Universal Transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*. 3

Dehghani, M., Gouws, S., Vinyals, O., Uszkoreit, J. and Kaiser, Ł. (2019). Universal Transformers. In *International Conference on Learning Representations (ICLR)*. 2, 3

Dey, N., Zhang, B. C., Noci, L., Li, M., Bordelon, B., Bergsma, S., Pehlevan, C., Hanin, B. and Hestness, J. (2025). Don't be lazy: CompleteP enables compute-efficient deep transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*. 36

Filatov, O., Wang, J., Ebert, J. and Kesselheim, S. (2025). Optimal Scaling Needs Optimal Norm. *arXiv preprint arXiv:2510.03871*. 22, 36

Fritsch, F. N. and Butland, J. (1984). A method for constructing local monotone piecewise cubic interpolants. *SIAM Journal on Scientific and Statistical Computing*, **5** 300–304. 6, 23

Fu, R., Yang, Z., Zhang, J., Ma, J., Chen, H., Li, Y. and Chang, Y. (2026). Simply Stabilizing the Loop via Fully Looped Transformer. *arXiv preprint arXiv:2605.18797*. 3, 15

Gao, Z., Chen, Y., Xiao, Y., Yang, X., Tao, R., Zhou, J. and Dai, B. (2026). Loop the Loopies! *arXiv preprint arXiv:2607.16051*. 3, 4

Geiping, J., McLeish, S., Jain, N., Kirchenbauer, J., Singh, S., Bartoldson, B. R., Kailkhura, B., Bhatele, A. and Goldstein, T. (2025). Scaling up Test-Time Compute with Latent Reasoning: A Recurrent Depth Approach. *arXiv preprint arXiv:2502.05171*. 2, 3, 4, 5, 21

Giannou, A., Rajput, S., Sohn, J.-y., Lee, K., Lee, J. D. and Papailiopoulos, D. (2023). Looped Transformers as Programmable Computers. *arXiv preprint arXiv:2301.13196*. 3

He, D., Li, P., Chang, D., Meng, Q., Yin, L. and Liu, S. (2026). Looping Beyond Twice: A Scalable Recipe for Looped Mixture-of-Experts. *arXiv preprint arXiv:2610.01153*. https://arxiv.org/abs/2610.01153 3, 4

Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., de Las Casas, D., Hendricks, L. A., Welbl, J., Clark, A., Hennigan, T., Noland, E., Millican, K., van den Driessche, G., Damoc, B. et al. (2022). Training Compute-Optimal Large Language Models. *arXiv preprint arXiv:2203.15556*. 3, 5, 11, 13, 26, 34

Huang, B., Shi, C., Chen, J., Wen, S., Liu, Z., Xing, E. and Ma, X. (2026). Towards Looped Models Done Right, Part II: Rethinking at Fixed Points. *arXiv preprint arXiv:2610.06833*. https://arxiv.org/abs/2610.06833 2, 3

Jordan, K., Jin, Y., Boza, V., You, J., Cesista, F., Newhouse, L. and Bernstein, J. (2024). Muon: An optimizer for hidden layers in neural networks. https://kellerjordan.github.io/posts/muon/. 5, 22, 27

Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J. and Amodei, D. (2020). Scaling Laws for Neural Language Models. *arXiv preprint arXiv:2001.08361*. 3, 26

Krajewski, J., Ludziejewski, J., Adamczewski, K., Pióro, M., Krutul, M., Antoniak, S., Ciebiera, K., Król, K., Odrzygóźdź, T., Sankowski, P., Cygan, M. and Jaszczur, S. (2024). Scaling Laws for Fine-Grained Mixture of Experts. *arXiv preprint arXiv:2402.07871*. 3

Lan, Z., Chen, M., Goodman, S., Gimpel, K., Sharma, P. and Soricut, R. (2019). ALBERT: A Lite BERT for Self-supervised Learning of Language Representations. *arXiv preprint arXiv:1909.11942*. 3, 34

Lee, R., Biloki, J., Hu, E. J. and May, J. (2026). Sparse Layers are Critical to Scaling Looped Language Models. *arXiv preprint arXiv:2605.09165*. 3, 4, 22

Li, H., Zheng, W., Wang, Q., Ding, Z., Wang, H., Wang, Z., Xuyang, S., Ding, N., Zhou, S., Zhang, X. and Jiang, D. (2025a). Predictable Scale: Part II, Farseer: A Refined Scaling Law in Large Language Models. *arXiv preprint arXiv:2506.10972*. 3

Li, H., Zheng, W., Wang, Q., Zhang, H., Wang, Z., Xuyang, S., Fan, Y., Ding, Z., Wang, H., Ding, N., Zhou, S., Zhang, X. and Jiang, D. (2025b). Predictable Scale: Part I, Step Law – Optimal Hyperparameter Scaling Law in Large Language Model Pretraining. *arXiv preprint arXiv:2503.04715*. 22, 36

Marchenko, A., Bezrukov, V., Kashurin, O., Fedorova, I., Bocharov, D., Shakhvalieva, Y., Tikhonova, M. and Ternovskii, V. (2026). Closing the Loop: Practical Training Recipes for Looped Language Models. *arXiv preprint arXiv:2610.00673*. 3

McLeish, S., Li, A., Kirchenbauer, J., Kalra, D. S., Bartoldson, B. R., Kailkhura, B., Schwarzschild, A., Geiping, J., Goldstein, T. and Goldblum, M. (2025). Teaching Pretrained Language Models to Think Deeper with Retrofitted Recurrence. *arXiv preprint arXiv:2511.07384*. 2, 3

Pearce, T. and Song, J. (2024). Reconciling Kaplan and Chinchilla Scaling Laws. *Transactions on Machine Learning Research*. 3

Penedo, G., Kydlíček, H., Ben Allal, L., Lozhkov, A., Mitchell, M., Raffel, C., Von Werra, L. and Wolf, T. (2024). The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale. *arXiv preprint arXiv:2406.17557*. 5, 22

Porian, T., Wortsman, M., Jitsev, J., Schmidt, L. and Carmon, Y. (2024). Resolving Discrepancies in Compute-Optimal Scaling of Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*. 3

Prairie, H., Novack, Z., Berg-Kirkpatrick, T. and Fu, D. Y. (2026). Parcae: Scaling Laws For Stable Looped Language Models. *arXiv preprint arXiv:2604.12946*. 2, 3, 4, 5, 6, 10, 13, 15, 21, 26, 27

Ren, L., Liu, Y., Shen, Y. and Chen, W. (2026). Rethinking Language Model Scaling under Transferable Hypersphere Optimization. *arXiv preprint arXiv:2603.28743*. 22, 35, 36

Saunshi, N., Dikkala, N., Li, Z., Kumar, S. and Reddi, S. J. (2025). Reasoning with Latent Thoughts: On the Power of Looped Transformers. In *International Conference on Learning Representations (ICLR)*. 3

Schwethelm, K., Rueckert, D. and Kaissis, G. (2026). How Much Is One Recurrence Worth? Iso-Depth Scaling Laws for Looped Language Models. *arXiv preprint arXiv:2604.21106*. 2, 4, 5, 8, 9, 10, 13, 14, 15, 21, 22, 25, 26, 27, 35

Tallec, C. and Ollivier, Y. (2017). Unbiasing Truncated Backpropagation Through Time. *arXiv preprint arXiv:1705.08209*. 3

Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., Bikel, D., Blecher, L., Ferrer, C. C., Chen, M., Cucurull, G. et al. (2023). Llama 2: Open Foundation and Fine-Tuned Chat Models. *arXiv preprint arXiv:2307.09288*. 22

Videau, M., Youbi-Idrissi, B., Lopez-Paz, D. and Ahuja, K. (2026). Skaling: Chinchilla's Exponents Meet Kaplan's Coupling. *arXiv preprint arXiv:2608.07222*. 3, 4, 5

Wang, G., Li, J., Sun, Y., Chen, X., Liu, C., Wu, Y., Lu, M., Song, S. and Yadkori, Y. A. (2025). Hierarchical Reasoning Model. *arXiv preprint arXiv:2506.21734*. 3

Wang, G., Liu, C., Wang, C., Zhou, C., Sun, Y., Wu, Y., Zhen, S., Scimeca, L. and Abbasi Yadkori, Y. (2026a). HRM-Text: Efficient Pretraining Beyond Scaling. *arXiv preprint arXiv:2605.20613*.
https://arxiv.org/abs/2605.20613 2, 3

Wang, S., Zhang, G., Luo, K., Wu, Y., Liu, S., Liu, J., Huang, W., Yan, S. and Li, J. (2026b). SMELT: Scaling Laws for Compute-Matched MoE Looped Transformers. *arXiv preprint arXiv:2609.01343*. 3, 4

Wen, K., Dang, X., Lyu, K., Ma, T. and Liang, P. (2025). Fantastic pretraining optimizers and where to find them 2.1: Hyperball optimization. https://tinyurl.com/muonh. 5, 22

Williams, R. J. and Peng, J. (1990). An Efficient Gradient-Based Algorithm for On-Line Training of Recurrent Network Trajectories. *Neural Computation*, **2** 490–501. 3

Yang, G., Hu, E. J., Babuschkin, I., Sidor, S., Liu, X., Farhi, D., Ryder, N., Pachocki, J., Chen, W. and Gao, J. (2021). Tensor Programs V: Tuning Large Neural Networks via Zero-Shot Hyperparameter Transfer. In *Advances in Neural Information Processing Systems (NeurIPS)*. 22

Yang, G., Yu, D., Zhu, C. and Hayou, S. (2023). Tensor Programs VI: Feature Learning in Infinite-Depth Neural Networks. *arXiv preprint arXiv:2310.02244*. 36

Yang, L., Lee, K., Nowak, R. and Papailiopoulos, D. (2024). Looped Transformers are Better at Learning Learning Algorithms. In *International Conference on Learning Representations (ICLR)*. 3

Zeitoun, A., Torroba-Hennigen, L. and Kim, Y. (2026). Hyperloop Transformers. *arXiv preprint arXiv:2604.21254*.
https://arxiv.org/abs/2604.21254 3

Zhu, R.-J., Wang, Z., Hua, K., Zhang, T., Li, Z., Que, H., Wei, B., Wen, Z., Yin, F., Xing, H., Li, L., Shi, J., Ma, K., Li, S., Kergan, T. et al. (2025). Scaling Latent Reasoning via Looped Language Models. *arXiv preprint arXiv:2510.25741*. 3, 4, 26

Contents

1 Introduction 2

2 Related work 3

3 Preliminaries 4

4 A scaling law for forward and backward loops 5
- 4.1 Fitting and validating the scaling law . . . 5
- 4.2 Illustrating the scaling law with effective parameters . . . 8

5 Truncate or not? 9
- 5.1 Fixed forward loops: truncation can be optimal . . . 9
- 5.2 Free forward-loop count: full BPTT is optimal . . . 10

6 Allocating compute among loops, data, and model size 11
- 6.1 Fixed model size: loops versus data . . . 12
- 6.2 Free model size: the law favors no looping . . . 14
- 6.3 Two fully trained loops are competitive with fewer parameters . . . 14

7 Conclusion 15

A Model architecture and experimental setup 21
- A.1 Model architecture . . . 21
- A.2 Experimental setup . . . 21

B The law in detail 22
- B.1 What the runs show before any law is fitted . . . 22
- B.2 Empirical looping coefficient . . . 23
- B.3 Form and fit . . . 24
- B.4 Why a bounded gain . . . 25
- B.5 Runs at sixteen loops . . . 26

C Training compute and allocation 26
- C.1 Training compute . . . 26
- C.2 Switch points . . . 27
- C.3 The runs at a fixed size . . . 27
- C.4 Allocation identities and their assumptions . . . 28
  - C.4.1 Fixed forward-loop count: optimize size, data, and backward loops . . . 28
  - C.4.2 Joint forward–backward allocation at equal compute . . . 30
  - C.4.3 Fixed model size: allocate loops and data . . . 31
  - C.4.4 Free model size: compare joint allocations . . . 33

D Compute-optimal pairs and the untied model 34

E Learning rate and batch size 35

F The law across the scaling suite 36

A Model architecture and experimental setup

A.1 Model architecture

Figure 12 draws the model of §3 for the loop setting $(4, 2)$. In a training step (Figure 12a), the prelude maps the tokens to $e = s_0$, every loop reads e again through the input injection, and the state s_{T-B} entering the last B loops is treated as a constant. The gradient therefore reaches the coda, the last B loops and, through the injection, the prelude; the first $T - B$ loops run forward only, and evaluation runs all T loops. Each loop (Figure 12b) applies W_{inj} to $[\,e\,;\,s_{t-1}\,]$, the ℓ_r pre-norm transformer layers of R and an RMSNorm, with the same weights in every loop. At 2 FLOPs per parameter and token for a forward pass and 4 for a backward pass (Figure 12c), the prelude and coda cost 6, each forward loop 2 and each backward loop 4 more, which gives approximately $N(3 + T + 2B)$ FLOPs per token under our parameter split, before attention and output costs (C.1); the optimizer update adds a cost per step (C.3).

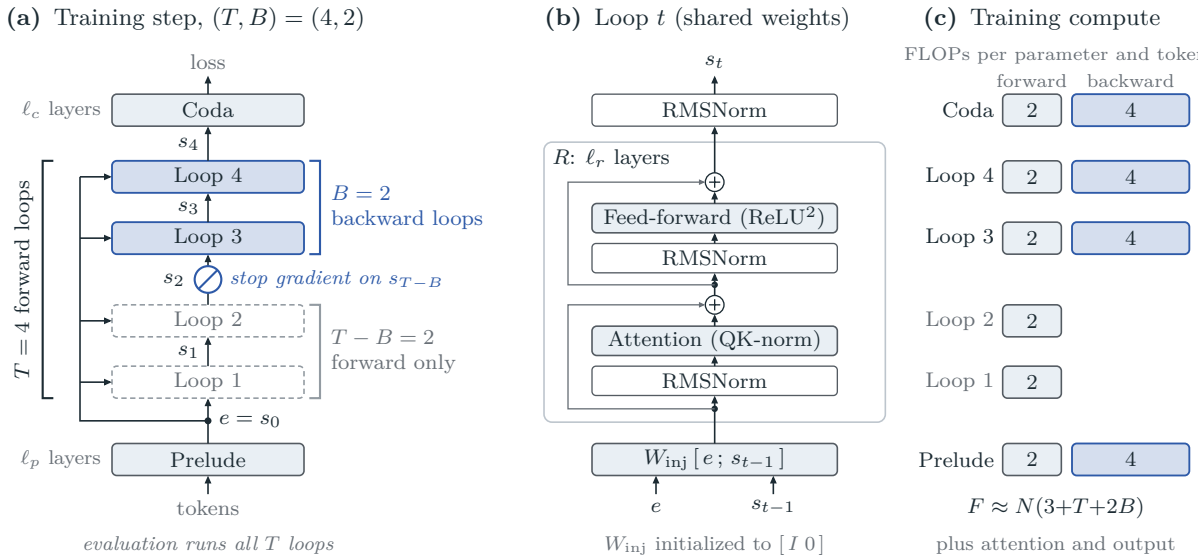


Figure 12 | **The looped transformer, drawn for** $(T, B) = (4, 2)$**. (a)** One training step: a stop gradient on s_{T-B}, so the first $T - B$ loops (dashed) run forward only. **(b)** One loop, (3.1); all loops share its weights. **(c)** Training FLOPs per parameter and token of each part (C.1).

A.2 Experimental setup

Models. We take the architecture and the training recipe of Schwethelm et al. (2026), the reference recipe below, which builds on the prelude–recurrent–coda layout of Huginn (Geiping et al., 2025) (§3). Its blocks use pre-norm attention with query–key normalization and rotary position embeddings, head dimension 128, a squared-ReLU feed-forward layer of width $4d$, no biases, and untied input and output embeddings. We change only what is held fixed. Schwethelm et al. (2026) hold the executed depth fixed, so that more loops mean fewer unique layers; we hold the unique parameters fixed, as Prairie et al. (2026) do, so that a loop setting changes the compute and not the model. We train five model sizes, from 58M to 919M non-embedding parameters (Table 3); width and depth grow together, and the recurrent block with the input injection holds 51% of the non-embedding parameters at four sizes and 56% at 120M. Short size names round N; including the untied input and output embeddings, total parameter counts run from about 107M to about 1.06B. We quote budgets per parameter as D/N.

Loop settings and budgets. The scaling suite uses fifteen loop settings. Ten core settings, $(1,1)$; $(2,1)$ and $(2,2)$; $(4,1)$, $(4,2)$, $(4,3)$ and $(4,4)$; and $(8,2)$, $(8,4)$ and $(8,8)$, cover $T \in \{1,2,4,8\}$ forward loops, full and truncated BPTT, and ratios B/T from $\frac{1}{4}$ to 1. Five more settings extend them: $(8,1)$, and $T = 16$ with $B = 1$, 4, 8 and 16 at three sizes and short budgets (Appendices B.5 and F). The part of the training compute that depends on the loops grows with $T + 2B$, since a backward loop costs twice a forward loop (§C.1); $(4,1)$ thus costs as much per token as $(2,2)$, and $(8,2)$ as much as $(4,4)$. Token budgets D are given in units of $D_0 = 2^{30} \approx 1.07 \times 10^9$ tokens. We group runs by their target token budget; actual token counts may differ slightly because of batch rounding. Predictions use the actual token count D. An experimental configuration is a choice of (N, D, T, B); a run is one training execution of that configuration. Each configuration contributes one accepted validation loss to the analysis.

Data and evaluation. We train on the 100B-token sample of FineWeb-Edu (Penedo et al., 2024), the corpus of the looped scaling study of Lee et al. (2026), in a pre-shuffled release,[3] tokenized with the Llama 2 tokenizer (Touvron et al., 2023) of the reference recipe (Schwethelm et al., 2026) into sequences of 2,048 tokens. We report cross-entropy in nats per token on the last two shards of the release, which no run trains on, after removing the documents whose exact text also appears in the training data (the same web page often occurs in several crawls).

Optimization. We also follow the reference recipe (Schwethelm et al., 2026) for optimization. MuonH, a variant of Muon that keeps each weight matrix at a fixed Frobenius norm (Jordan et al., 2024; Wen et al., 2025), trains the matrices of the transformer blocks, and AdamW trains the embedding, output and normalization parameters, both without weight decay. The MuonH update adds 0.06–5.7% to the compute of the forward and backward passes; the analysis leaves it out, and the measured compute we report includes it (§C.1). The peak learning rates follow the reference's transfer rule (Ren et al., 2026; Schwethelm et al., 2026; Yang et al., 2021), a power law in the batch and training tokens with the token exponent re-estimated for our budgets, and the global batch grows as $64\sqrt{D/D_0}$ sequences, following the square-root scaling of the loss-optimal batch in prior work (Bergsma et al., 2025; Filatov et al., 2025; Li et al., 2025b) (§E). The learning rate warms up over the first 2% of steps and decays with a cosine to 10% of its peak; gradients are not clipped.

Runs and compute. The results in this paper come from 592 training runs with 2.7×10^{22} FLOPs of training compute, counted as in §C.1 with the optimizer update: the 385 runs behind the 343 configurations of the scaling suite (§F; a configuration's loss is the mean of its accepted runs) and four more runs at 480M in Figure 10, the 21 runs of the compute-optimal pairs and the 36 of the untied comparison (§D), and the 66 learning-rate and 124 batch-size runs of Figures 14 and 15; 44 runs belong to two of these groups and are counted once. Counting cancelled runs, runs replaced by reruns, pilot runs and earlier calibration sweeps that the paper does not report, the study trained more than 1,100 runs with about 4.8×10^{22} FLOPs.

B The law in detail

This appendix details the measured patterns behind the law, its fit and validation, the bounded gain, and the runs at sixteen loops.

B.1 What the runs show before any law is fitted

The measured losses separate the effects of forward and backward loops without a fitted law (§F). Forward loops that receive no gradient add almost nothing at first: at fixed B, doubling T from B

[3] https://huggingface.co/datasets/karpathy/fineweb-edu-100b-shuffle

to $2B$ changes the loss by less than 0.007 nats in all 24 size–budget combinations trained in all ten core settings, while doubling it again, from $2B$ to $4B$, lowers the loss in all 24. Spending the same compute per token on backward loops is better: $(2, 2)$ beats $(4, 1)$ and $(4, 4)$ beats $(8, 2)$ in all 24, by 0.023 nats on average for $(2, 2)$ against $(4, 1)$. Backward loops in turn have diminishing returns: with full BPTT, every doubling of $T = B$, from 1 to 8, lowers the loss, by less each time, in all 24 combinations where all four settings were trained.

B.2 Empirical looping coefficient

The looping coefficient $g(T, B)$ in (4.1) rescales the recurrent block's parameter count. To estimate it from the runs, we first find the non-looped model size that matches a looped run's measured loss at the same token budget. This defines the empirical effective size, which we then convert to a looping coefficient using the run's parameter counts.

Let $d = D_{\text{target}}/D_0$ denote a target budget level, where D_{target} is the target token count and $D_0 = 2^{30}$ tokens. At each d, we collect the accepted non-looped $(T, B) = (1, 1)$ configurations and order their actual parameter counts and measured losses as $(n_j, \ell_j)_{j=1}^{m_d}$, where $n_1 < \cdots < n_{m_d}$. Each configuration contributes its accepted loss once, following the same aggregation rules as the scaling-law fit. We require at least three distinct sizes and strictly decreasing losses, $\ell_1 > \cdots > \ell_{m_d}$, as holds for the eligible reference groups. We interpolate log size as a function of loss:

$$\phi_d(\ell) \coloneqq \text{PCHIP}[(\ell_j, \log n_j)_{j=m_d,\ldots,1}]\,(\ell), \qquad \ell \in [\ell_{m_d}, \ell_1]. \tag{B.1}$$

Here PCHIP denotes shape-preserving piecewise cubic Hermite interpolation (Fritsch and Butland, 1984), with reference points ordered by increasing loss. The interpolant in (B.1) passes through each measured point, $\phi_d(\ell_j) = \log n_j$, and preserves monotonicity between points. This loss-to-size conversion uses only the non-looped measurements, independent of the fitted coefficients of (4.1).

For a looped configuration i, let L_i be its measured loss, d_i its target budget, and $N_{\text{once},i}, N_{\text{rec},i} > 0$ its once-run and recurrent counts; when its reference group is eligible and $L_i \in [\ell_{m_{d_i}}, \ell_1]$, define

$$\widehat{N}_{\text{eff},i} \coloneqq \exp[\phi_{d_i}(L_i)], \qquad \widehat{g}_i \coloneqq \frac{\widehat{N}_{\text{eff},i} - N_{\text{once},i}}{N_{\text{rec},i}}. \tag{B.2}$$

In (B.2), we subtract the once-run parameters and divide by the recurrent block's size, as in (4.1). Each empirical coefficient depends on the run's size and token budget; $g(T, B)$ is the law's shared prediction. We report an estimate only when the reference group is eligible and the measured loss lies within its observed range, without extrapolation. Budget matching uses the target level d, keeping batch-rounded budgets in the same group. Actual consumed tokens differ from the target token budget by less than 0.2% in these runs; we leave these differences uncorrected.

For example, the 239.24M non-embedding-parameter run at $T = B = 2$ and $D = 4D_0$ has loss 2.4272 nats per token. The reference interpolation gives $\widehat{N}_{\text{eff}} = 345.20$M, hence $\widehat{g} = (345.20 - 117.98)/121.26 \simeq 1.874$, using its own once-run and recurrent counts in millions of parameters.

For each setting (T, B), let $\mathcal{I}_{T,B}$ be the set of eligible size–budget configurations. Figure 3 shows the median of $\{\widehat{g}_i : i \in \mathcal{I}_{T,B}\}$, with the 25th and 75th percentiles as the interquartile range. Configurations have equal weight; for sorted estimates $x_{(1)}, \ldots, x_{(n)}$, a percentile at probability u linearly interpolates adjacent order statistics at rank $1 + (n - 1)u$. Figure 6 uses the same loss-matched estimates, with total-size ratios shown where indicated. The surface figure includes 163 eligible configurations with $T \leq 8$: 121 from the fitting set and 42 from the separately held-out set. Reference configurations may also come from either set, and estimates that use the same references are dependent. These figures therefore describe agreement with the runs rather than

independent validation of the law. The interquartile ranges show variation across sizes and budgets, not confidence intervals. §B.3 reports the separate holdout evaluations in loss units.

Figure 2 uses the same estimator on the 150 eligible configurations with $T \in \{2, 4, 8\}$ and $1/4 \leq r = B/T \leq 1$. Markers show medians and bars show minima and maxima. For each T, we fit an intercept a_T and slope s_T by ordinary least squares with equal weight per cell:

$$(\hat{a}_T, \hat{s}_T) = \arg\min_{a,s} \sum_{i:T_i=T} [a + s \log_2 r_i - \log_2 \hat{g}_i]^2 . \tag{B.3}$$

In (B.3), the fitted slopes are 0.823, 0.446 and 0.173 for $T = 2, 4, 8$, respectively; these lines summarize the empirical trend across the 150 configurations rather than a separate scaling law.

B.3 Form and fit

The looping coefficient is

$$g(T, B) = \exp[c(1 - T^{-p} B^{-q})], \qquad N_{\text{eff}} = N_{\text{once}} + g(T, B) N_{\text{rec}}.$$

The normalization is exact: $g(1, 1) = 1$, so $(T, B) = (1, 1)$ gives $N_{\text{eff}} = N$. For $c, p, q > 0$, $g(T, B)$ increases in T and in B toward e^c. The fitted ceiling of the looping coefficient is $e^c \simeq 3.04$; it is an extrapolation, not an observed asymptotic limit. With full BPTT the looping coefficient is $g(T, T) = \exp[c(1 - T^{-(p+q)})]$; for $B < T$ it is smaller by the factor $\exp[-cT^{-(p+q)}(r^{-q} - 1)]$, $r = B/T$, which approaches one as T grows.

We fit 230 configurations spanning fifteen loop settings. The part of the scaling suite that the law is fitted on covers five model sizes on four diagonals of nearly constant tokens per parameter (about 9, 18, 37 and 74), giving the sizes a common budget range. It contains 194 configurations in the eleven settings with at most eight loops (a few at the largest budgets were not run) and 32 with $T = 16$. Four configurations from two compute-matched pairs complete the fitting set, adding sizes 79M and 162M to the five sizes of the scaling suite. Each configuration enters once through its accepted loss, aggregating repeated runs and seeds and removing duplicate exports. The other 117 configurations of the scaling suite, mostly at shorter or longer budgets, enter no fit and serve as separate tests. Predictions use actual consumed tokens and exact architecture parameter counts. The fixed normalizations are $N_0 = 2.4 \times 10^8$ and $D_0 = 2^{30}$.

We minimize equally weighted squared residuals in nats per token. All nine coefficients are refitted on every training fold using 64 independent uniform random starts, retaining the lowest-cost converged solution. The trust-region reflective solver uses an analytic Jacobian, Jacobian scaling, at most 4000 evaluations, and tolerances ftol = xtol = gtol = 10^{-12}. Seeds are 20261010 plus the deterministic fold index. No full-data coefficients initialize a held-out fold, and no learned preprocessing, weighting or hyperparameter tuning is used.

In optimization coordinates, $E \in [0, 3]$, $\log A, \log H \in [-10, 15]$, $\alpha, \beta, \kappa \in [0.01, 3]$, $p, q \in [0.001, 10]$, and $\log c \in [\log 0.001, \log 10]$. Initial ranges are $E \in [0.3, 1.2]$, $\log A, \log H \in [0.5, 2.5]$, $\alpha, \beta \in [0.25, 0.65]$, $\kappa \in [0.15, 0.45]$, $p, q \in [0.1, 2]$, and $\log c \in [\log 0.2, \log 2]$. The power-law gain of §B.4 uses the same ranges for the first six coefficients and $\phi_T, \phi_B \in [-5, 5]$, initialized in $[0.05, 0.8]$. All 264 fits of each law converge with full-rank Jacobians (the largest-size holdout reaches $E = 0$).

Validation and aggregation. Residuals are prediction minus measurement. Point LOO holds out one entire accepted configuration. Size folds hold out all combinations at the selected model size. Budget folds group the target level D_{target}/D_0, retaining batch-rounded versions together. Setting folds hold out an entire (T, B) pair. Each family covers all 230 configurations. Point LOO trains on 229 configurations and tests on the remaining one, repeating this for all 230. Table 7 reports pooled

errors, which weight held-out configurations equally. Every fold converges and none is dropped. §F overlays the law on the 226 fitted configurations of the scaling suite; the four fitted configurations from the compute-matched pairs are not shown there.

Table 7 | Pooled errors in nats/token over all 230 configurations; all 264 fits converge (one at a bound).

Evaluation	Fits	RMSE	MAE	Bias	Max. error
In-sample	1	6.14×10^{-3}	4.91×10^{-3}	-3.73×10^{-16}	2.04×10^{-2}
One configuration out	230	6.40×10^{-3}	5.11×10^{-3}	-3.30×10^{-6}	2.10×10^{-2}
One model size N out	7	7.48×10^{-3}	5.89×10^{-3}	-2.16×10^{-4}	2.58×10^{-2}
One token budget D out	11	6.42×10^{-3}	5.12×10^{-3}	-3.42×10^{-5}	2.08×10^{-2}
One (T, B) setting out	15	8.67×10^{-3}	6.74×10^{-3}	8.86×10^{-4}	2.63×10^{-2}

Table 8 | The law against the power-law gain of prior work (Schwethelm et al., 2026): $g(T, B) = T^{\phi_T} B^{\phi_B}$ in place of the bounded gain, which reduces to T^{ϕ} with full BPTT, everything else of (4.1) unchanged, fitted on the same configurations with the same folds and solver. Pooled RMSE in nats per token.

Evaluation	Bounded gain	Power-law gain
In-sample	6.14×10^{-3}	1.31×10^{-2}
One configuration out	6.40×10^{-3}	1.36×10^{-2}
One model size N out	7.48×10^{-3}	1.40×10^{-2}
One token budget D out	6.42×10^{-3}	1.35×10^{-2}
One (T, B) setting out	8.67×10^{-3}	1.65×10^{-2}

Table 9 | Looping coefficient and total effective size; total size uses recurrent share $f = 0.507$.

Setting	$g(T, B)$	N_{eff}/N
$(1, 1)$	1.00000	1.00000
$(2, 1)$	1.18330	1.09294
$(2, 2)$	1.91023	1.46150
$(4, 1)$	1.36498	1.18505
$(4, 2)$	2.04945	1.53209
$(4, 4)$	2.50361	1.76236
$(8, 2)$	2.17552	1.59601
$(8, 4)$	2.57832	1.80024
$(8, 8)$	2.80330	1.91431
$(16, 16)$	2.93895	1.98308

B.4 Why a bounded gain

Prior scaling laws for looped models let the gain of looping grow as a power of the loop count, T^{ϕ} (Schwethelm et al., 2026). We extend this form to $T^{\phi_T} B^{\phi_B}$ and substitute it for $g(T, B)$, keeping the rest of (4.1) unchanged. This form is the limit of our gain as $c \to \infty$ with $cp \to \phi_T$ and $cq \to \phi_B$; under full BPTT it reduces to T^{ϕ} with $\phi = \phi_T + \phi_B$. We fit both laws on the same configurations using the same folds and solver. The power-law gain is less accurate in every evaluation in Table 8, with nearly twice the error when a (T, B) setting is held out (1.65×10^{-2} against 8.67×10^{-3} nats); it keeps growing where the runs level off, and held out at sixteen loops its error is three times that of the bounded gain (§B.5). Our gain rises from $g(1, 1) = 1$ toward e^{c}; with full BPTT and $p+q = 1$ it is the reciprocal exponential gain $\exp[c(1 - 1/T)]$, and free exponents separate the ceiling from the rate of approach.

The derivatives distinguish the empirical assumption from its algebraic consequences. At fixed coefficients, with $s = p + q$, the total derivatives below follow the full-BPTT path $B = T$:

$$\frac{\partial \log g}{\partial T} = cpT^{-p-1}B^{-q} > 0, \qquad \frac{\partial \log g}{\partial B} = cqT^{-p}B^{-q-1} > 0,$$
$$\frac{\mathrm{d}}{\mathrm{d}T}g(T,T) = csT^{-s-1}g(T,T) > 0,$$
$$\frac{\mathrm{d}^2}{\mathrm{d}T^2}g(T,T) = csT^{-s-2}g(T,T)\,[csT^{-s} - (s+1)].$$

For the fitted coefficients, $cs < s + 1$, so the full-BPTT gain is concave for $T \geq 1$. Concavity is not automatic for every positive parameter choice. The effective size is an equivalence in validation loss, not in stored knowledge (Zhu et al., 2025).

B.5 Runs at sixteen loops

The fit contains 32 configurations at $T = 16$, across 120M, 240M and 480M, with backward counts 1, 4, 8 and 16 and target token budgets from $2D_0$ to $8D_0$. Holding out this whole row leaves 198 training configurations and gives RMSE 0.0115 nats, against 0.0340 for the power-law gain of prior work, which keeps growing beyond the loop counts it is fitted on. The harder extrapolation is the row without looping, $T = 1$: RMSE 0.0172 (0.0207 for the power-law gain), the largest forward-row error. The four sixteen-loop configurations of 480M at $2D_0$ lie outside the fitting set.

C Training compute and allocation

We first define the training-compute cost, then discuss the fixed-size switch points and their experimental checks. §C.4 gives the allocation derivations for prescribed forward loops, joint loop counts, and fixed or free model size.

C.1 Training compute

Training compute counts the FLOPs of the forward and backward passes. We use the usual convention of 2 FLOPs per parameter and token for the forward pass and 4 for the backward pass (Hoffmann et al., 2022; Kaplan et al., 2020). The prelude and coda run forward and backward once per token. Each forward loop runs the recurrent block forward once, and each backward loop adds a backward pass through it, which costs twice as much. The compute of the passes per token, using the approximately equal parameter split, is

$$F(T,B) \;\approx\; N\,(3 + T + 2B) + F_{\text{attn}}(T,B) + F_{\text{out}}, \tag{C.1}$$

where F_{attn} counts the attention products of every executed layer and F_{out} the output projection.[4] The attention products of the recurrent block follow the same count, one unit per forward loop and two more per backward loop, so every loop-dependent term grows with $T+2B$, as in the accounting of truncated BPTT by Prairie et al. (2026) and Schwethelm et al. (2026); for a given model, $(2,2)$ and $(4,1)$ therefore cost the same per token, as do $(4,4)$ and $(8,2)$.

[4] We use actual layer counts for numerical compute accounting. Each transformer layer has $12d^2$ matrix parameters and input injection has $2d^2$, so the exact matrix contribution under our counting convention is $72d^2(\ell_p + \ell_c) + 4d^2(T + 2B)(6\ell_r + 1)$. With width d, sequence length S and vocabulary size V (32,064 with padding): $F_{\text{attn}} = 4Sd\,[\,3(\ell_p + \ell_c) + (T + 2B)\,\ell_r\,]$ and $F_{\text{out}} = 6Vd$. Embedding lookup, normalization, and activations are omitted.

Table 10 | Model size fixed: the training compute at which the best loop setting changes, given as the tokens per parameter that the model without looping trains on at that compute. Among all fifteen trained settings, four of them with $T = 16$, only full BPTT reaches the frontier, apart from $(4, 3)$, which leads briefly between $(2, 2)$ and $(4, 4)$ by at most 0.002 nats. †Beyond the compute of the runs of that size. ‡Involves $T = 16$, trained at three sizes and short budgets only.

Best setting changes	60M	120M	240M	480M	1B
$(1, 1) \to (2, 2)$	16	17	16	15	14
$(2, 2) \to (4, 4)$	400	413	374†	345†	312†
$(4, 4) \to (8, 8)$	9,622†	9,648†	8,605†	7,833†	7,025†
$(8, 8) \to (16, 16)$	210,087‡	204,851‡	182,247‡	164,533‡	146,710‡

For a model of size N we write this compute per token as $F_N(T, B) = u_N + v_N(T + 2B)$, where u_N collects the prelude, coda and output and $v_N > 0$ one forward pass through the recurrent block with its attention, so that D training tokens cost

$$C \simeq F_N(T, B)\, D. \tag{C.2}$$

At fixed N and D, the compute depends on the loop counts only through $T + 2B$. (5.1) is its leading term, without the attention and output costs.

The optimizer update is negligible. MuonH also spends FLOPs on each update: five Newton–Schulz iterations per weight matrix (Jordan et al., 2024), a cost F_{opt} per step that depends on the model but not on T or B, so that a run of n_{step} steps costs

$$C = F(T, B)\, D + F_{\text{opt}}\, n_{\text{step}}. \tag{C.3}$$

Across our runs the update adds 0.06–5.7% to the compute of the passes. It cannot change the split between forward and backward loops, since it does not depend on either; leaving it out moves the fixed-size switch points of §6.1 by less than 0.5%, and it gives the models with more trainable parameters, the untied model and the model without looping at its compute-optimal size, 0.5–0.75% more compute, or about 0.001 nats in loss (§D). Like prior scaling studies (Prairie et al., 2026; Schwethelm et al., 2026), our analysis therefore omits it and uses (C.2); the measured compute in figures, tables and §A.2 includes it unless stated otherwise.

C.2 Switch points

At a fixed size, each setting traces a loss curve in C, and the best setting at each compute lies on their lower envelope (Table 10). Full BPTT reaches the envelope almost everywhere: among all fifteen trained settings, four of them with $T = 16$, the best setting at every compute is $(1, 1)$, $(2, 2)$, $(4, 4)$, $(8, 8)$ or $(16, 16)$, apart from a narrow compute range between $(2, 2)$ and $(4, 4)$ where $(4, 3)$ leads by at most 0.002 nats on our integer menu (§6.1). Truncation lowers the compute per token but gives up the gain of the loops it does not train, and at equal compute per token a setting that trains all of its fewer loops is the better buy, as the runs show directly (§B.1). Measured in the tokens per parameter that the model without looping trains on at the same compute, two loops become the best choice at about 14–17, four overtake two at about 312–413, and eight overtake four at about 7,000–9,600 (sixteen pay only beyond 147,000–210,000).

C.3 The runs at a fixed size

The runs confirm the first switch and the role of the trained loops (Figure 10; Figure 1c previews the 120M slice). Beyond about 30 tokens per parameter, $(2, 2)$ ends below the model without looping

at every point we measured, by about 0.01 nats at 50 tokens per parameter and by 0.025 at 200. Loops without gradient do not win: $(4, 1)$, which costs as much per token as $(2, 2)$ but trains only one of its four loops, stays above the model without looping up to 200 tokens per parameter. At 60M, 120M and 240M, where the runs reach several hundred tokens per parameter, $(4, 4)$ overtakes $(2, 2)$, as the law predicts, but already at about 200 tokens per parameter rather than at the law's 374–413. At one compute per size far into this range, that of $(4, 4)$ at $32D_0$ for 60M and 240M and at $24D_0$ for 120M (1,291, 533 and 335 tokens per parameter of the model without looping), the runs read 2.514, 2.389 and 2.247 for the model without looping, 2.479, 2.358 and 2.223 for $(2, 2)$, and 2.460, 2.345 and 2.218 for $(4, 4)$: $(4, 4)$ is the lowest, 0.019, 0.012 and 0.006 nats below $(2, 2)$, while $(4, 3)$ at the same compute trails $(4, 4)$ by 0.010, 0.008 and 0.006 and $(8, 8)$ lies above it at all three sizes. At 480M the same compute is reached at 174 tokens per parameter, before the switch: the model without looping reads 2.141, $(2, 2)$ 2.121 and $(4, 4)$ 2.124, so $(2, 2)$ and $(4, 4)$ are level and both stay below the model without looping. The runs with $T = 16$, trained at $2D_0$–$8D_0$ (§B.5), lie well below the compute where eight or sixteen loops pay and stay above $(4, 4)$ at equal compute.

C.4 Allocation identities and their assumptions

We follow the allocation regimes of Sections 5.1 to 6.2: first prescribe T, then optimize the split between T, B, and finally allocate loops, data, and model size. The prescribed-T and free-size results use the leading matrix-FLOP approximation (5.1); the joint-loop and fixed-size calculations use (C.2), with the attention and output costs. All omit the optimizer update (§C.1).

C.4.1 Fixed forward-loop count: optimize size, data, and backward loops

Lemma C.1 (Optimal backward-loop allocation)**.** *Fix the integer forward-loop count $T \geq 1$ and training-compute budget $C > 0$. Suppose the loss follows (4.1), with positive fitted coefficients $E, A, H, \alpha, \beta, \kappa, c, p, q$ and normalization scales $N_0, D_0 > 0$. Let $N = N_{\text{once}} + N_{\text{rec}}$ be the parameter count. Assume the recurrent block holds a fixed fraction $\rho \in [0, 1]$ of parameters as N varies ($N_{\text{rec}} = \rho N$, $N_{\text{once}} = (1 - \rho)N$). Under the leading-FLOP approximation $C = 6DN_{\text{comp}}$, for each $B \in \{1, \dots, T\}$ let $L^*_{T,B}(C)$ be the minimum loss over $N, D > 0$ at budget C; then*

$$\operatorname*{arg\,min}_{B \in \{1,\dots,T\}} L^*_{T,B}(C) = \operatorname*{arg\,min}_{B \in \{1,\dots,T\}} \chi(T, B), \qquad \chi(T, B) = \frac{N_{\text{comp}}}{N_{\text{eff}}}. \tag{C.4}$$

The minimizing backward counts are independent of C.

Proof. Fix B. Substituting ρ into the effective and compute-equivalent parameter counts gives

$$\frac{N_{\text{eff}}}{N} = 1 - \rho + \rho g(T, B), \qquad \chi(T, B) = \frac{1 - \rho + \rho(T + 2B)/3}{1 - \rho + \rho g(T, B)} > 0.$$

Thus χ is independent of N, and N_{eff} is a positive constant multiple of N. Optimizing over $N > 0$ is therefore equivalent to optimizing over $x = N_{\text{eff}}/N_0 > 0$. Define $y = D/D_0$ and $\bar{C} = C/(6N_0D_0)$. The budget is $C = 6D\chi N_{\text{eff}}$, so $\bar{C} = \chi xy$ and $y = \bar{C}/(\chi x)$. Since $L = E + J^\kappa$ with $\kappa > 0$, minimizing L is equivalent to minimizing J; eliminating y gives

$$J(x) = Ax^{-\alpha} + H\left(\frac{\chi x}{\bar{C}}\right)^{\beta}.$$

To optimize size, differentiate with respect to $\log x$, holding T, B, C fixed. With $X = Ax^{-\alpha}$ and $Y = H(\chi x/\bar{C})^\beta$, the derivatives are

$$\frac{\mathrm{d}J}{\mathrm{d}\log x} = -\alpha X + \beta Y, \qquad \frac{\mathrm{d}^2 J}{\mathrm{d}(\log x)^2} = \alpha^2 X + \beta^2 Y > 0.$$

Thus the objective is strictly convex in $\log x$. It also diverges as $x \to 0$ or $x \to \infty$, so its stationary point is the unique global minimum. The stationary balance $\alpha X = \beta Y$ gives

$$\alpha A x^{-\alpha} = \beta H \left(\frac{\chi}{\bar{C}}\right)^{\beta} x^{\beta} \quad \Longrightarrow \quad x^{\alpha+\beta} = \frac{\alpha A}{\beta H}\left(\frac{\bar{C}}{\chi}\right)^{\beta}.$$

Taking the positive root and substituting into $y = \bar{C}/(\chi x)$ yields

$$\begin{aligned} x^* &= \left(\frac{\alpha A}{\beta H}\right)^{1/(\alpha+\beta)} \left(\frac{\bar{C}}{\chi}\right)^{\beta/(\alpha+\beta)}, \\ y^* &= \left(\frac{\beta H}{\alpha A}\right)^{1/(\alpha+\beta)} \left(\frac{\bar{C}}{\chi}\right)^{\alpha/(\alpha+\beta)}. \end{aligned} \tag{C.5}$$

The optimal size and token count are recovered as $N^* = N_0 x^*/[1-\rho+\rho g(T,B)]$ and $D^* = D_0 y^*$. For the optimal loss, use $Y^* = (\alpha/\beta)X^*$ at the stationary point and substitute x^*:

$$\begin{aligned} J^* &= \frac{\alpha+\beta}{\beta} A (x^*)^{-\alpha} \\ &= \frac{\alpha+\beta}{\beta} A^{\beta/(\alpha+\beta)} \left(\frac{\beta H}{\alpha}\right)^{\alpha/(\alpha+\beta)} \left(\frac{\bar{C}}{\chi}\right)^{-\alpha\beta/(\alpha+\beta)}. \end{aligned}$$

Define the positive constants

$$\eta = \frac{\kappa\alpha\beta}{\alpha+\beta}, \qquad K = \left[\frac{\alpha+\beta}{\beta} A^{\beta/(\alpha+\beta)} \left(\frac{\beta H}{\alpha}\right)^{\alpha/(\alpha+\beta)}\right]^{\kappa}.$$

Restoring the outer exponent of the loss law then gives

$$L^*_{T,B}(C) - E = K \bar{C}^{-\eta} \chi(T,B)^{\eta}. \tag{C.6}$$

Both $K\bar{C}^{-\eta}$ and $\eta > 0$ are common to all candidate backward counts. The optimized loss is therefore strictly increasing in χ, which proves (C.4), including any ties. The factor involving C is common to all backward counts, so their ranking is independent of the budget when the admissible set of B is fixed. This includes $\rho = 0$, for which $\chi = 1$ and all backward counts tie. □

For a restricted menu, minimize over its available backward counts. Figure 13 evaluates χ for every integer B at $T = 2, 4, 8, 16$ (Table 5 lists the optima).

Separate laws for the contour comparison. To check the shared looping coefficient $g(T,B)$, we fit $L = E_s + [A_s(N/N_0)^{-\alpha_s} + H_s(D/D_0)^{-\beta_s}]^{\kappa_s}$ separately to each setting s at $T = 4, 8$ in Figure 7. The contours use compute from (5.1) with $N_{\text{rec}}/N \simeq 0.507$; their minimum-compute points trace the compute-optimal frontiers. These are in-sample predictions of the separate laws.

Choosing the backward-loop count. For a continuous relaxation, an interior optimum balances the relative increase in compute-equivalent size against that in loss-equivalent size. The ratio is independent of model size, so we can differentiate it at fixed $N_{\text{once}}, N_{\text{rec}}$. By definition,

$$\frac{\partial N_{\text{comp}}}{\partial B} = \frac{2}{3} N_{\text{rec}}, \qquad \frac{\partial N_{\text{eff}}}{\partial B} = cqT^{-p}B^{-q-1} g(T,B) N_{\text{rec}}.$$

Differentiating $\log \chi = \log N_{\text{comp}} - \log N_{\text{eff}}$ and setting the derivative to zero yields

$$\frac{2BN_{\text{rec}}}{3N_{\text{comp}}} = cqT^{-p}B^{-q}\frac{g(T,B)N_{\text{rec}}}{N_{\text{eff}}}. \tag{C.7}$$

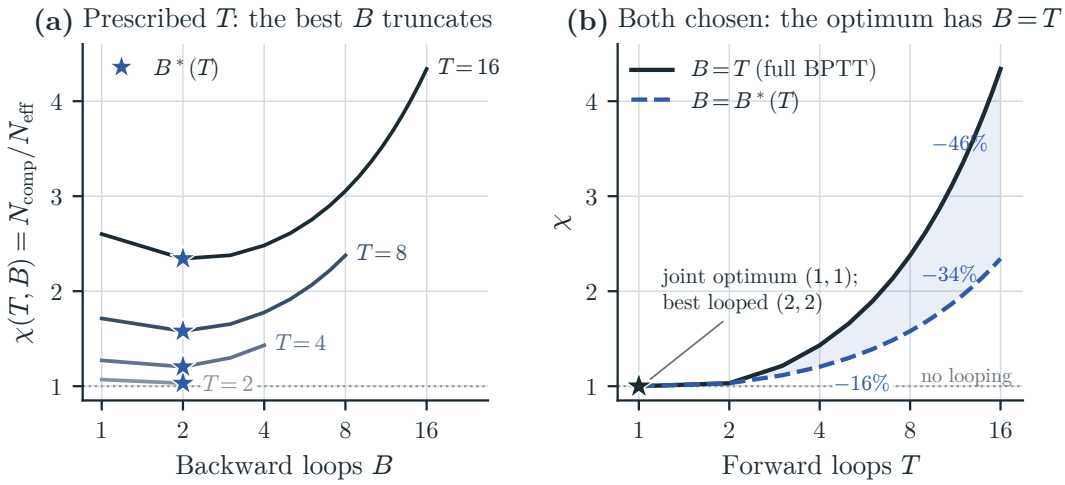


Figure 13 | **Truncation can be favored at a prescribed T, but not when T and B are chosen jointly.** The law's $\chi = N_{\text{comp}}/N_{\text{eff}}$; lower χ means lower compute-optimal loss. (a) Stars: the best B at each T. (b) Percentages: compute saved by the best B over $B = T$ at equal loss.

This is an implicit equation for a continuous candidate, as both parameter counts depend on B; boundary values $B = 1, T$ are also checked, and the discrete optimum evaluates χ at each integer.

The equation also gives an explicit asymptotic form when $N_{\text{rec}} > 0$. Suppose $0 < p < 1$ and $q > 0$, as in the fitted law, and consider large T with $B \ll T$. Then $g(T, B) \to e^c$, $N_{\text{comp}} \sim TN_{\text{rec}}/3$, and $N_{\text{eff}} \to N_{\text{once}} + e^c N_{\text{rec}}$. Substituting these limits into (C.7) and rearranging gives

$$B^*(T) \sim \left[\frac{cq}{2}\frac{e^c N_{\text{rec}}}{N_{\text{once}} + e^c N_{\text{rec}}}\right]^{1/(1+q)} T^{(1-p)/(1+q)}.$$

The exponent lies between zero and one, so this solution grows while $B^*/T \to 0$, matching the assumed limit. This is a large-T prediction; finite loop counts require discrete optimization.

C.4.2 Joint forward–backward allocation at equal compute

Lemma C.2 (Optimal joint forward–backward allocation)**.** *Fix an architecture with $N_{\text{rec}} > 0$ and a token count $D > 0$. Suppose the loss follows (4.1), with positive fitted coefficients $E, A, H, \alpha, \beta, \kappa$, c, p, q and positive normalization scales N_0, D_0. Let T and B be continuous variables. For any $S \geq 3$, consider the equal-compute problem under (C.2):*

$$\underset{T,B\in\mathbb{R}}{\text{minimize}} \quad L(N, D, T, B) \qquad \textit{subject to} \quad T + 2B = S, \qquad 1 \leq B \leq T.$$

If $q \geq 2p$, its unique solution is $T^* = B^* = S/3$, corresponding to full BPTT.

Proof. At fixed architecture and token count, (C.2) depends on the loop counts only through $S = T + 2B$, including the attention products. Thus every feasible split has the same compute. Since the loss is strictly decreasing in N_{eff} and $N_{\text{rec}} > 0$, minimizing it is equivalent to maximizing $g(T, B)$. Using $g(T, B) = \exp[c(1 - T^{-p}B^{-q})]$ with $c > 0$, this is equivalent to minimizing $T^{-p}B^{-q}$, or its logarithm. Substituting $T = S - 2B$ gives $1 \leq B \leq S/3$ and the strictly convex objective

$$h(B) = -p\log(S - 2B) - q\log B, \qquad h''(B) = \frac{4p}{(S-2B)^2} + \frac{q}{B^2} > 0.$$

When $q \geq 2p$, the constraint $B \leq T$ gives

$$h'(B) = \frac{2p}{T} - \frac{q}{B} \leq \frac{2p-q}{T} \leq 0.$$

The derivative is strictly negative whenever $B < T$, including when $q = 2p$. Hence the unique minimum is at $B = S/3$, where $T = B$. For $S = 3$, this is also the only feasible split. □

The replacement preserves N, D and strictly improves any truncated setting, so the conclusion also holds when either or both are optimized, provided the replacement is admissible. It does not require fixed parameter proportions across model sizes. Including the optimizer update does not change the result, since its cost is independent of T, B.

The condition is stronger than $q > p$: a backward loop costs twice a forward loop, so the comparison is q versus $2p$. Our adopted fit satisfies $q > 2p$. For integer or restricted loop menus, $S/3$ may be unavailable, so the continuous dominance argument does not replace the discrete check (C.8).

C.4.3 Fixed model size: allocate loops and data

With N fixed, size cannot rebalance against data, and the loop setting determines how many tokens the compute buys: $D_N(C; T, B) = C/F_N(T, B)$ by (C.2). §6.1 compares the fifteen observed settings $\mathcal{S}$ at their respective token budgets:

$$(T^*, B^*) \in \arg\min_{(T,B)\in\mathcal{S}} L(N, D_N(C; T, B), T, B), \qquad D^* = D_N(C; T^*, B^*). \tag{C.8}$$

The switch points quoted in §6.1 are computed with the measured compute, optimizer update included; leaving it out moves them by less than 0.5% (§C.1).

For continuous loop counts, Lemma C.2 establishes the preference for full BPTT when $q \geq 2p$, as in our adopted fit. We therefore set $B = T$ and first characterize the interior balance between loops and tokens, then derive its large-compute limit.

Lemma C.3 (Optimal loop–token balance at fixed model size)**.** *Fix an architecture with $N_{\text{once}} \geq 0$ and $N_{\text{rec}} > 0$, and a compute budget $C > 0$. Suppose the loss follows (4.1), with positive fitted coefficients $E, A, H, \alpha, \beta, \kappa, c, p, q$ and positive normalization scales N_0, D_0. Let $B = T \geq 1$ be continuous and use (C.2), so that $F_N(T, T) = u_N + 3v_N T$ with fixed $u_N, v_N > 0$ and $D = C/F_N(T, T)$. Write $s = p + q$ and define*

$$N_{\text{eff}}(T) = N_{\text{once}} + N_{\text{rec}} \exp[c(1 - T^{-s})],$$
$$X(T) = A\left(\frac{N_{\text{eff}}(T)}{N_0}\right)^{-\alpha}, \qquad Y(T) = H\left(\frac{D}{D_0}\right)^{-\beta}.$$

Throughout this derivation, a prime denotes a total derivative with respect to T along $B = T$, holding the architecture and C fixed. In particular,

$$N'_{\text{eff}}(T) \equiv \frac{\mathrm{d}N_{\text{eff}}(T)}{\mathrm{d}T} = N_{\text{rec}} cs\, g(T, T) T^{-s-1},$$
$$F'_N(T, T) \equiv \frac{\mathrm{d}}{\mathrm{d}T} F_N(T, T) = 3v_N.$$

Thus N'_{eff} measures how effective size changes with loop count (N is fixed), and $F'_N(T, T)$ includes both loop counts as $B = T$ grows. Every interior optimum satisfies

$$\alpha X \frac{N'_{\text{eff}}}{N_{\text{eff}}} = \beta Y \frac{F'_N(T, T)}{F_N(T, T)}. \tag{C.9}$$

Equivalently, the stationary points are parameterized by

$$C(T) = D_0 \left[\frac{3\beta H v_N}{\alpha A} F_N(T,T)^{\beta-1} \left(\frac{N_{\text{eff}}}{N_0} \right)^{\alpha} \frac{N_{\text{eff}}}{N'_{\text{eff}}} \right]^{1/\beta}, \qquad D(T) = \frac{C(T)}{F_N(T,T)}. \tag{C.10}$$

Proof. Since $\kappa > 0$, minimizing loss is equivalent to minimizing J; substituting the budget gives

$$J(T; N, C) = X(T) + Y(T) = A \left(\frac{N_{\text{eff}}}{N_0} \right)^{-\alpha} + H \left(\frac{D_0 F_N(T,T)}{C} \right)^{\beta}.$$

Increasing T raises the effective size and lowers X, but reduces the token budget and raises Y. At fixed N, C, differentiating the token budget gives

$$D' \equiv \frac{\mathrm{d}}{\mathrm{d}T} \left(\frac{C}{F_N(T,T)} \right) = -\frac{C F'_N(T,T)}{F_N(T,T)^2}, \qquad \frac{D'}{D} = -\frac{F'_N(T,T)}{F_N(T,T)}.$$

Consequently, the derivatives of the two loss terms are

$$X' \equiv \frac{\mathrm{d}X}{\mathrm{d}T} = -\alpha X \frac{N'_{\text{eff}}}{N_{\text{eff}}}, \qquad Y' \equiv \frac{\mathrm{d}Y}{\mathrm{d}T} = \beta Y \frac{F'_N(T,T)}{F_N(T,T)}.$$

Setting $J' = X' + Y' = 0$ gives (C.9), and $N'_{\text{eff}} = N_{\text{rec}} cs\, g(T,T) T^{-s-1} > 0$ with $F'_N = 3v_N$ yields

$$\alpha A \left(\frac{N_{\text{eff}}}{N_0} \right)^{-\alpha} \frac{N'_{\text{eff}}}{N_{\text{eff}}} = 3\beta H v_N \left(\frac{D_0}{C} \right)^{\beta} F_N(T,T)^{\beta-1}.$$

All factors are positive, so solving for C gives (C.10). □

This is a necessary condition for an interior optimum; boundary values and all allowed integer settings must also be compared. On the finite menu, added compute goes to tokens until a deeper setting reaches a lower predicted loss, at which point the token budget drops.

Lemma C.4 (Asymptotic loop–token allocation)**.** *Suppose the loss follows (4.1), with positive fitted coefficients $E, A, H, \alpha, \beta, \kappa, c, p, q$ and positive normalization scales N_0, D_0. Use the fixed architecture and compute model of Lemma C.3, and allow all continuous $T \geq 1$ with no upper bound. Let $T^*(C)$ be any global minimizer along $B = T$, and let $D^*(C) = C/F_N(T^*, T^*)$. As $C \to \infty$, there are constants $K_T, K_D > 0$, independent of C, such that*

$$T^*(C) \sim K_T C^{\frac{\beta}{\beta+p+q}}, \qquad D^*(C) \sim K_D C^{\frac{p+q}{\beta+p+q}}. \tag{C.11}$$

Proof. For each finite C, the continuous objective J tends to infinity as $T \to \infty$, since its data term grows without bound, so a global minimizer exists. Moreover, $T^*(C) \to \infty$ as $C \to \infty$: for any fixed $M \geq 1$, every $T \leq M$ has $J(T; N, C) \geq X(M)$, whereas $J(2M; N, C) \to X(2M) < X(M)$. No minimizer stays in $[1, M]$ at large compute; it is eventually interior and satisfies (C.10).

Writing $s = p + q$, the large-T limits are

$$N_{\text{eff}} \longrightarrow N_{\text{once}} + e^c N_{\text{rec}},$$
$$N'_{\text{eff}} \sim N_{\text{rec}} cs e^c T^{-s-1}, \qquad F_N(T,T) \sim 3v_N T.$$

The bracket in (C.10) is thus asymptotic to a positive constant times $T^{\beta-1} T^{s+1} = T^{\beta+s}$, so $C(T)$ is asymptotic to a positive constant times $T^{(\beta+s)/\beta}$. Evaluating this relation at $T^*(C)$ and inverting gives the first part of (C.11), and $D^*(C) \sim C/(3v_N T^*(C))$ gives the second, recovering (6.3). □

The current coefficients give exponents approximately 0.238 and 0.762 ((6.4) rounds them to 0.24 and 0.76). These are asymptotic consequences of the fitted law along $B = T$, not a refit to measured optima. The dominance argument above applies within this law and compute model; its extrapolation to unbounded depth is not empirically established. Finite-depth overheads and allowed loop counts determine the practical switches.

C.4.4 Free model size: compare joint allocations

The effective-size comparison in §6.2 follows directly from the fitted gain. Write $s = p+q$. At fixed token count, replacing a fully trained T-loop model with a non-looped model of size $N_{\text{once}} + TN_{\text{rec}}$ preserves the leading compute and increases effective size by $[T - g(T,T)]N_{\text{rec}}$. For the adopted coefficients, $g(2,2) \simeq 1.910 < 2$ and

$$\frac{\mathrm{d}}{\mathrm{d}T} \log \frac{g(T,T)}{T} = \frac{csT^{-s} - 1}{T} < 0 \qquad (T \geq 2),$$

since $cs\, 2^{-s} \simeq 0.585 < 1$ and $s > 0$. Hence $g(T,T) < T$ for all integers $T \geq 2$. This holds for integer loop counts (the continuous extension has $g(T,T) > T$ just above $T = 1$, as $cs \simeq 1.400 > 1$).

When T is also free, reuse the optimal size and data allocation from (C.5) for each candidate (T,B), under the leading matrix-FLOP approximation (5.1) and a fixed recurrent fraction. Let $L^*_{T,B}(C)$ denote the minimum loss over positive, continuous N, D, and $N^*_{T,B}(C)$, $D^*_{T,B}(C)$ its allocations. Write the budget as $C/\chi = 6N_{\text{eff}}D$. The objective in N_{eff}, D is the non-looped loss law, with the budget of a non-looped model at C/χ. Its optimal size and data are thus $N^*_{\text{eff}} = N^*_{1,1}(C/\chi)$ and $D^*_{T,B}(C) = D^*_{1,1}(C/\chi)$; recovering unique size via $N^* = (N/N_{\text{eff}})N^*_{\text{eff}}$ gives

$$\begin{aligned} L^*_{T,B}(C) &= L^*_{1,1}\left(\frac{C}{\chi(T,B)}\right), \\ N^*_{T,B}(C) &= \left(\frac{N}{N_{\text{eff}}}\right)_{T,B} N^*_{1,1}\left(\frac{C}{\chi(T,B)}\right), \\ D^*_{T,B}(C) &= D^*_{1,1}\left(\frac{C}{\chi(T,B)}\right). \end{aligned} \tag{C.12}$$

The ratio $(N/N_{\text{eff}})_{T,B}$ is evaluated at the indicated loop setting and is independent of N when parameter proportions are fixed. These identities are exact within the stated approximation, with two consequences. First, by (C.5) every loop setting has $N^* \propto C^{\beta/(\alpha+\beta)}$ and $D^* \propto C^{\alpha/(\alpha+\beta)}$ (exponents 0.562 and 0.438 with the fitted coefficients), so the optimal pair minimizes $\chi(T,B)$ at every compute. Second, among the fifteen settings $(1,1)$ minimizes χ and $(2,2)$ is the best looped setting (Table 5; Figure 13b). For $(2,2)$, $N_{\text{eff}} \simeq 1.462N$, $N_{\text{comp}} \simeq 1.507N$ and $\chi \simeq 1.031$, so (C.12) predicts equal loss at 3.1% more compute and 31.6% fewer parameters than $(1,1)$.

For the compute-premium comparison, let a candidate loop setting receive λC while $(1,1)$ receives C. (C.6) and $\chi(1,1) = 1$ give

$$\frac{L^*_{T,B}(\lambda C) - E}{L^*_{1,1}(C) - E} = \left(\frac{\lambda}{\chi(T,B)}\right)^{-\eta}.$$

Since $\eta > 0$, the looped loss is smaller when $\lambda > \chi(T,B)$, and (C.5) gives $N^*_{1,1}(C) \propto C^{\beta/(\alpha+\beta)}$, so

$$\frac{N^*_{T,B}(\lambda C)}{N^*_{1,1}(C)} = \left(\frac{N}{N_{\text{eff}}}\right)_{T,B} \left(\frac{\lambda}{\chi(T,B)}\right)^{\beta/(\alpha+\beta)}.$$

For $(2,2)$, 10% more compute predicts lower loss with about 29.0% fewer non-embedding parameters. At equal compute ($\lambda = 1$), subtracting the baseline loss gives the gap ($\eta = \kappa\alpha\beta/(\alpha+\beta) \simeq 0.0559$):

$$L^*_{T,B}(C) - L^*_{1,1}(C) = \left[L^*_{1,1}(C) - E\right]\left[\chi(T,B)^{\eta} - 1\right].$$

For $(2,2)$ the bracketed multiplier is about 1.72×10^{-3}, or 0.172% of the non-looped model's excess loss above E, rather than a constant gap in nats, making the compute convention consequential.

Limits of the scale-independent ratios. The identities describe an empirical loss law under a specified compute approximation; they do not establish a universal property of looped models. Size or data bounds can put an optimum on a boundary. A size-dependent recurrent share or the attention, output, and optimizer terms breaks the simple $6\chi N_{\text{eff}}D$ factorization. Full-compute optimization therefore requires an explicit architectural scaling family and a fresh numerical envelope. Its results should be reported separately from the analytic ratios, using the same adopted fit.

D Compute-optimal pairs and the untied model

The main text (§6.3) compares $(2,2)$ with a compute-optimal model without looping while allowing model size and token count to vary. Below we give the full table for that comparison, followed by a controlled comparison at fixed tokens against an untied model with a separate block per loop.

Compute-optimal pairs. We trained $(1,1)$ and $(2,2)$ each at the size and token count that minimize its loss for the same training compute, with three seeds each (Table 11). At all three budgets, the looped model has 27–32% fewer non-embedding parameters and comes within $(1\text{–}8)\times10^{-3}$ nats of the model without looping. At 2.1×10^{19} FLOPs (2.0×10^{19} for the forward and backward passes alone), training the 344M $(2,2)$ model on 8% more tokens, and so with 8% more compute, lowers its loss to 2.3590, $(7.3\pm1.8)\times10^{-3}$ nats below the 471M model without looping (Figure 1d).

Table 11 | Compute-optimal pairs. Each model at the size that minimizes its loss for the compute; loss: mean of three seeds; gap: $(2,2)$ minus $(1,1)$, $\pm$ standard error; parameters: non-embedding counts (the models without looping are the 120M, 240M and 480M models of Table 3). Compute includes attention, output, and optimizer updates (C.3); the first three rows are matched in the compute of the forward and backward passes (1.5×10^{18}, 5.9×10^{18}, and 2.0×10^{19} FLOPs, or 9.6×10^{17}, 4.1×10^{18}, and 1.5×10^{19} under the leading-FLOP count $6N_{\text{comp}}D$ of Figure 9d; counting the optimizer update, $(1,1)$ spends 0.7% more, which corresponds to about 10^{-3} nats); the last row trains the 344M $(2,2)$ model on 8% more tokens (+8% compute) against the 2.1×10^{19} FLOP $(1,1)$ reference.

	$(1,1)$		$(2,2)$		
Compute (FLOPs)	Parameters	Loss	Parameters	Loss	Gap
1.6×10^{18}	115M	2.7098	79M	2.7177	$(7.9\pm1.9)\times10^{-3}$
6.0×10^{18}	239M	2.5215	162M	2.5253	$(3.8\pm1.8)\times10^{-3}$
2.1×10^{19}	471M	2.3663	344M	2.3677	$(1.4\pm1.7)\times10^{-3}$
2.2×10^{19} (+8%)	471M	2.3663	344M	2.3590	$(-7.3\pm1.8)\times10^{-3}$

The untied model. The untied model of a looped model is the same network in which each loop has its own copy of the recurrent block, input injection and normalization. For two loops it holds $N_{\text{once}}+2N_{\text{rec}}$ parameters, about 1.5 times as many, at the same compute per token. Tying layers to save parameters goes back to ALBERT (Lan et al., 2019), and Bae et al. (2025a) relax the tie of looped models converted from pretrained ones. Each copy is initialized like the looped model; with all copies set equal, the untied model reproduces the looped model's loss and logits exactly.

Design and results. Each size trains at about 18 tokens per parameter of the looped model, close to the compute-optimal ratio of Hoffmann et al. (2022): 60M at D_0, 120M at $2D_0$, 240M at $4D_0$ and 480M at $8D_0$, with the recipe of its $(2,2)$ run and three seeds per model. Alongside the untied model, we train $(2,2)$ at the same width and $(2,2)$ widened to about 80% of the untied model's non-embedding parameters, on the same tokens (Table 12). At the untied model's width the looped model trails by 9×10^{-3}–1.7×10^{-2} nats (1.3×10^{-2} averaged over the four sizes), against $(3\text{–}4)\times10^{-3}$ by the law, whose error at these $(2,2)$ runs is at most 4×10^{-3} nats. The wider looped model uses

Table 12 | **Comparison with the untied model.** At each size: the untied model, $(2, 2)$ at its width, and $(2, 2)$ widened to about 80% of its non-embedding parameters. Width with heads × head dimension; parameters: non-embedding; compute: compute per token of the forward and backward passes relative to the untied model, the same ratio as the training compute since all models of a size train on the same tokens; loss: mean of three seeds; − untied: paired by seed, mean ± standard error.

Size	Model	Width	Parameters	Compute	Loss	− untied
60M	untied	768 (6×128)	87M	1.00	2.7843	
	$(2, 2)$	768 (6×128)	58M	1.00	2.7931	$(+8.8 \pm 3.0) \times 10^{-3}$
	$(2, 2)$, wider	840 (7×120)	69M	1.15	2.7668	$(-1.75 \pm 0.19) \times 10^{-2}$
120M	untied	1024 (8×128)	180M	1.00	2.5809	
	$(2, 2)$	1024 (8×128)	115M	1.00	2.5982	$(+1.73 \pm 0.14) \times 10^{-2}$
	$(2, 2)$, wider	1152 (9×128)	146M	1.22	2.5681	$(-1.28 \pm 0.23) \times 10^{-2}$
240M	untied	1280 (10×128)	360M	1.00	2.4147	
	$(2, 2)$	1280 (10×128)	239M	1.00	2.4272	$(+1.25 \pm 0.08) \times 10^{-2}$
	$(2, 2)$, wider	1408 (11×128)	289M	1.18	2.4044	$(-1.03 \pm 0.17) \times 10^{-2}$
480M	untied	1664 (13×128)	709M	1.00	2.2565	
	$(2, 2)$	1664 (13×128)	471M	1.00	2.2700	$(+1.35 \pm 0.19) \times 10^{-2}$
	$(2, 2)$, wider	1824 (12×152)	566M	1.18	2.2511	$(-5.4 \pm 1.5) \times 10^{-3}$

15–22% more forward-and-backward compute per token than the untied model and reaches a lower loss by 5×10^{-3}–1.7×10^{-2} nats, in all twelve seed pairs.

E Learning rate and batch size

Our runs follow one learning-rate rule and one batch-size rule, both set by the token budget D (Table 13). We test each rule against sweeps that vary only the learning rate or only the batch, and call a value acceptable when its loss is within 0.25% of the optimum of its sweep, about a tenth of what $(4, 4)$ gains over $(1, 1)$ at the same model size and budget.

Table 13 | Batch size, number of steps and peak learning rate of the matrix parameters at each budget. The learning rate is (E.1) with $\gamma = 0.40$, with its ratio to the $\gamma = 0.32$ value rounded to a power of $2^{1/8}$; the AdamW learning rates are multiplied by the same ratio. Warmup takes 2% of the steps.

Budget D/D_0	$\frac{1}{2}$	1	2	4	8	16	32
Batch (sequences of 2,048 tokens)	48	64	88	128	176	256	352
Steps	5,464	8,192	11,920	16,384	23,832	32,768	47,664
Peak learning rate ($\times 10^{-3}$)	10.7	9.25	8.11	8.01	7.03	6.36	6.09

Learning-rate rule. We take the transfer rule of the reference recipe (Schwethelm et al., 2026) for the peak learning rate of the matrix parameters,

$$\eta \;=\; 0.014 \left(\frac{b}{128}\right)^{0.558} \left(\frac{D}{D_{\text{ref}}}\right)^{-\gamma}, \qquad D_{\text{ref}} = 9.83 \times 10^{8}, \tag{E.1}$$

where b is the global batch in sequences of 2,048 tokens; the AdamW learning rates scale with width, and those of the embedding and output also with batch. The reference recipe uses the token exponent $\gamma = 0.32$ of HyperP (Ren et al., 2026). On our models this exponent is too small: relative to (E.1) with $\gamma = 0.32$, the optimal learning rate keeps falling as D grows (Figure 14b). We thus re-estimate γ and, as our models vary in depth, test if a depth term is needed.

Token exponent: $\gamma = 0.40$. We swept the learning rate at the smallest and the largest model size, 60M at D_0, $4D_0$ and $16D_0$ and 1B at $4D_0$ and $16D_0$, each with $(4, 4)$ and $(4, 1)$, and fit a quadratic in $\log \eta$ to each sweep to locate its optimum (Figure 14a). The exponent that minimizes the rule's largest loss above the optimum is $\gamma = 0.406$ (95% interval 0.400–0.412), and we use $\gamma = 0.40$. The rule is then within 0.06% of the optimum in every sweep, the largest gap being 60M with $(4, 1)$ at $16D_0$, whereas $\gamma = 0.32$ ends up 0.45% above the optimum at 1B and $16D_0$.

No depth correction. Depth-wise transfer rules lower the learning rate as networks get deeper (Bordelon et al., 2023; Dey et al., 2025; Ren et al., 2026; Yang et al., 2023). For updates normalized as in MuonH, keeping the change of the network per step fixed requires $\eta \propto \ell^{-1/2}$ with residual branches scaled by $\ell^{-1/2}$, or $\eta \propto \ell^{-1}$ without that scaling, where ℓ is the number of executed layers (Ren et al., 2026, Theorem 3). Our recipe scales neither the residual branches nor the learning rate with depth, although our models get deeper both with size (20 to 40 executed layers from 60M to 1B at $T = 4$) and with the number of forward loops (12 to 36 at 60M). Yet the optimum does not follow the depth (Figure 14c–d): at D_0, the rule's learning rate is the best of $1/\sqrt{2}$, 1 and $\sqrt{2}$ times itself in all six sweeps, and the fitted optima stay within about 7% of the rule, where the trend HyperP measures without residual scaling would lower them by more than half. The optimal learning rate follows the training tokens, not the depth, so the rule needs no depth term.

Batch-size rule. We set the global batch to

$$b(D) \;=\; 64\sqrt{D/D_0} \text{ sequences}, \tag{E.2}$$

rounded to a ladder whose neighboring steps differ by about $\sqrt{2}$ (..., 32, 48, 64, 88, 128, 176, 256, 352). The loss-optimal batch grows about as $\sqrt{D}$, as in prior work (Bergsma et al., 2025; Filatov et al., 2025; Li et al., 2025b), and in eleven batch sweeps with $(4, 4)$ (60M at five budgets, 240M and 1B at three) the rule's batch is acceptable in every one (Figure 15a).

Prefactor. The loss is flat over about a factor of two in batch, so the prefactor is a choice between neighboring ladder steps. One step lower, $45\sqrt{D/D_0}$, suits 1B with full BPTT as well as 64 suits 60M, but truncated BPTT prefers larger batches: at 1B and $16D_0$, the best batch is 176 for $(4, 4)$ and 352 for $(4, 1)$. Over all 33 sweeps with at least three batch sizes, which cover all ten core settings, only the rule is acceptable everywhere: it stays within 0.17% of the best batch, while one step lower reaches 0.30% and one step higher 0.42% (Figure 15b). The rule therefore changes comparisons between loop settings by less than 0.004 nats.

F The law across the scaling suite

Figure 16 shows the validation loss of the 343 configurations of the scaling suite, one panel per loop setting. Filled markers denote the 226 configurations used to fit the law of §4; open markers denote the rest. The four fitted configurations from the compute-matched pairs (120M, 79M, 240M and 162M) are not drawn. The figure contains 290 configurations in the ten core settings, 17 with $(8, 1)$ and 36 with $T = 16$. Each configuration shows the mean loss of its accepted runs (385 in all; §A.2); the law is drawn over each size's measured budgets. The configurations span 1.1 to 1,291 tokens per parameter, while the fitted ones span 9 to 74. Per loop setting, the law's RMSE on the fitted configurations is 0.0031 to 0.0093 nats; on the rest it is 0.0200, as most of them lie below 5 or above 125 tokens per parameter (§7). Figure 4 shows the four full-BPTT settings at the 26 size–budget pairs where all four were trained: one set of coefficients tracks all four, with RMSE 0.0053 nats over the 68 fitted configurations in the figure (0.0148 over the 36 others).

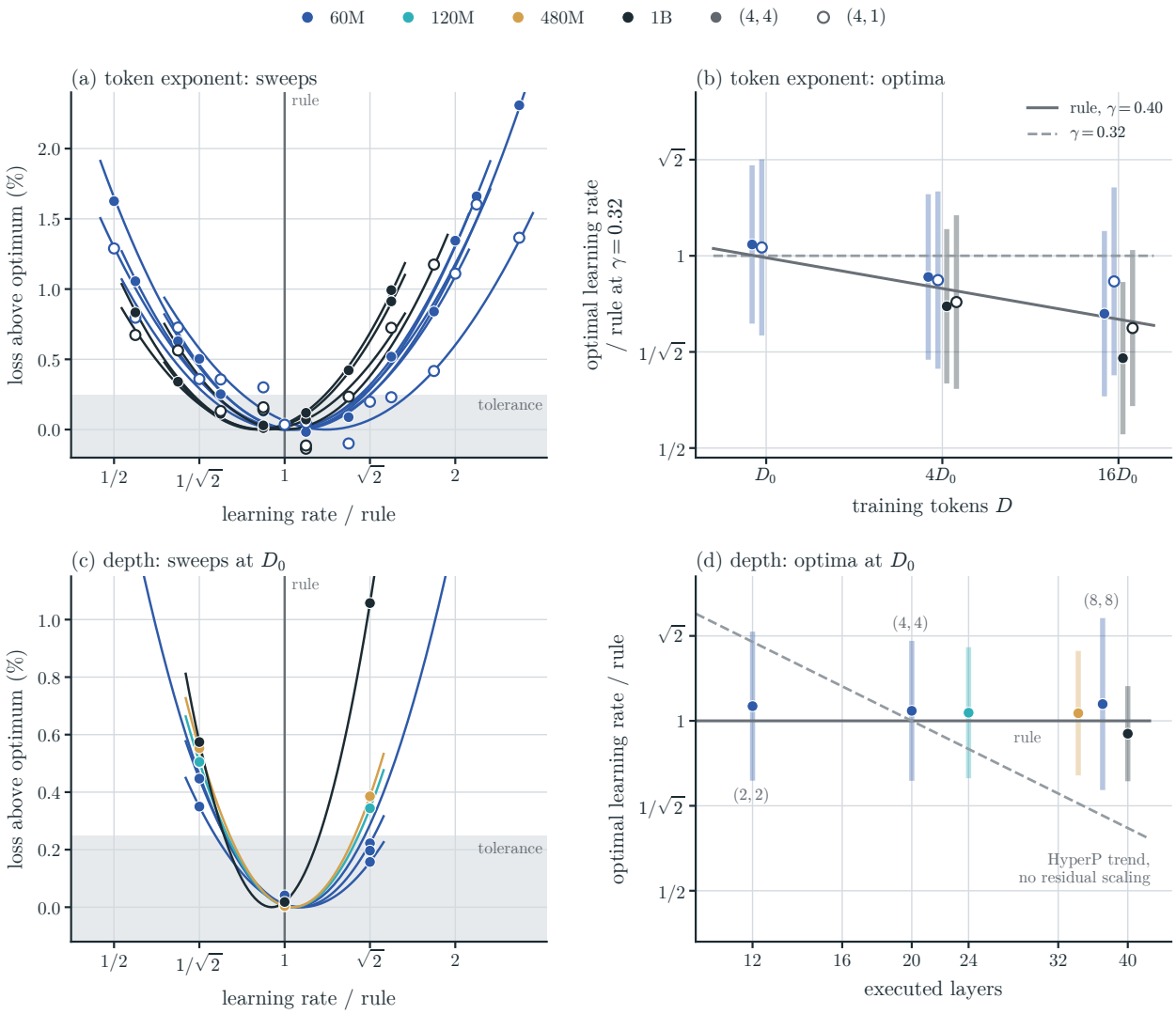


Figure 14 | Learning-rate rule ((E.1), $\gamma = 0.40$) against the measured optimum. Colors give the model size; filled markers: $(4, 4)$, open markers: $(4, 1)$. Solid gray lines: the rule; dashed lines: the alternative it is tested against; shading: the tolerance (0.25%). (a) Loss above the fitted optimum, in percent of the optimum loss, against the learning rate relative to the rule, for the ten token-exponent sweeps, with one quadratic in $\log \eta$ per sweep. (b) Fitted optima relative to (E.1) at $\gamma = 0.32$, with the learning rates within the tolerance (bars); dashed: $\gamma = 0.32$. (c) As (a), at D_0, for four sizes with $(4, 4)$ and for 60M with $(2, 2)$ and $(8, 8)$. (d) Optima of these sweeps against the number of executed layers; dashed: the trend HyperP measures without residual scaling (the optimum halves from 8 to 24 layers), drawn through the rule at 20 layers.

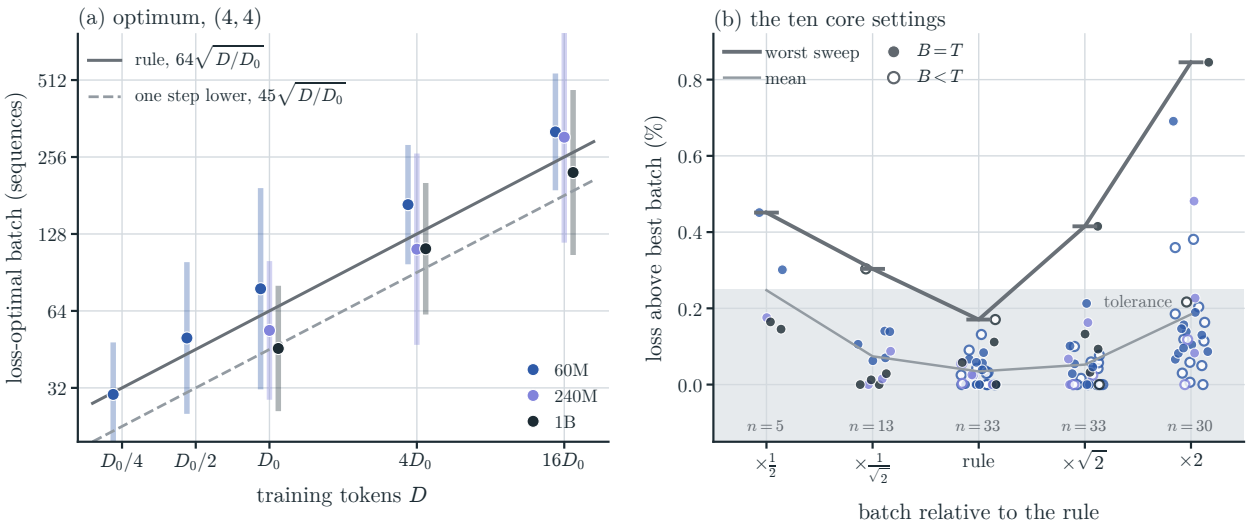


Figure 15 | Batch rule (E.2) against the measured optimum; colors give the model size. (a) Loss-optimal batch for $(4, 4)$, the vertex of a quadratic in log batch fitted to each sweep, with the batches within the tolerance (bars). Lines: the rule (solid) and one ladder step lower (dashed). (b) Loss above the best batch, in percent of the best loss, at up to two ladder steps from the rule, over the 33 sweeps with at least three batch sizes (filled: $B = T$; open: $B < T$; n: sweeps run at that batch). Thick line: worst sweep; thin line: mean; shading: the tolerance.

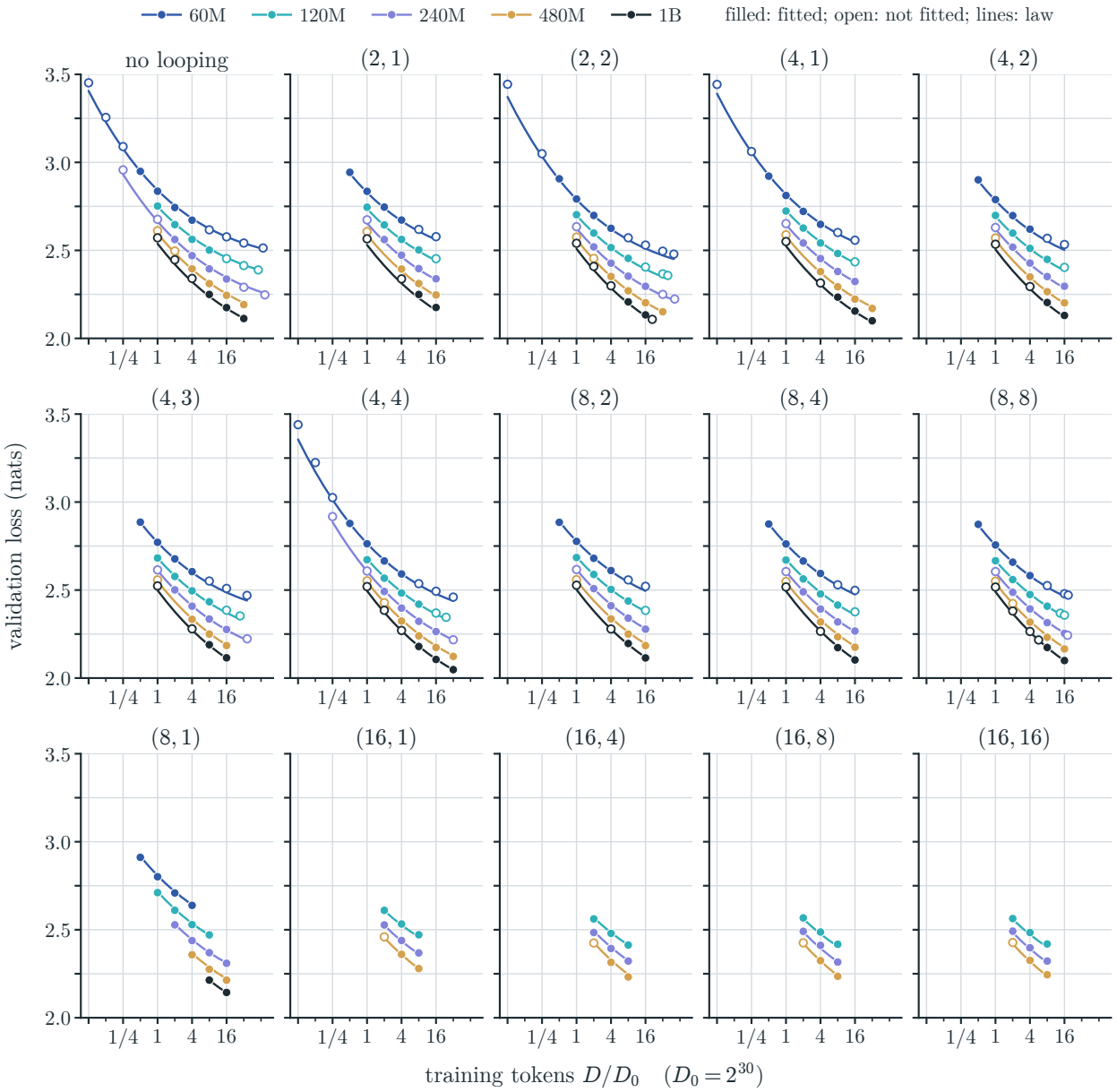


Figure 16 | Validation loss of every configuration of the scaling suite against training tokens ($D_0 = 2^{30}$), one panel per loop setting, colored by model size. Filled dots: configurations the law is fitted on; open dots: the other configurations; lines: the law (§4) over each size's measured budgets.